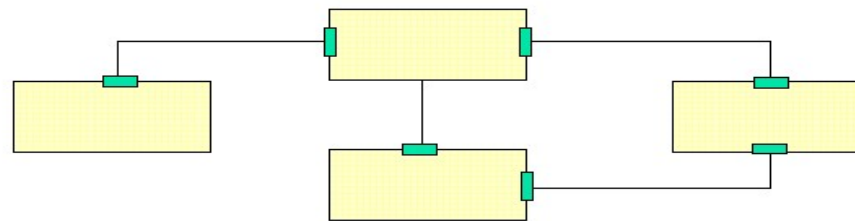


CONCEPTION ARCHITECTURALE



- Processus durant lequel on doit IMAGINER, PROPOSER une **architecture** pour satisfaire les spécifications (objectifs et contraintes)
- Décomposition en modules, et mise en évidence des frontières entre ces modules, structuration de l'ensemble
- Recherche de plusieurs solutions, comparaison et choix entre les différentes alternatives de conception
- A la fin de cette étape on doit disposer d'un modèle :
 - COMPLET
 - COHERENT
 - MAINTENABLE
 - TESTABLE
- Entrée
 - Document de spécification
- Tâches
 - Définir l'architecture du logiciel :
 - les composants de l'architecture
 - les interfaces (relations) entre les composants
- Sortie (Document produit)
 - document de conception globale (Architecture du logiciel à réaliser)

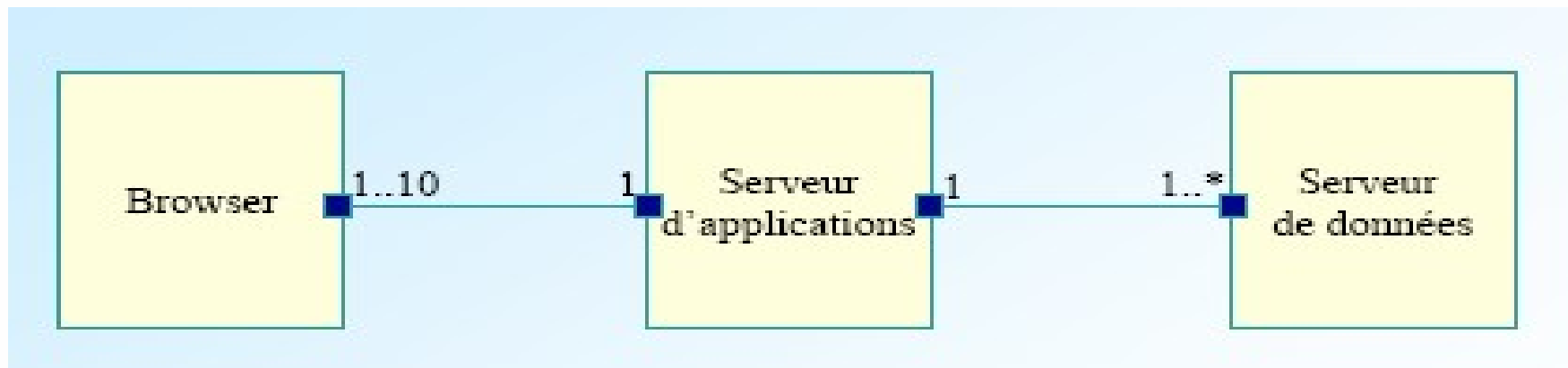
- L'architecture logicielle définit **la structure** (ou les structures) d'un logiciel qui comprend les éléments logiciels, leurs propriétés visibles et leurs relations.
- L'architecture correspond à l'organisation fondamentale d'un système informatique. Elle se définit par les composants nécessaires, leur composition entre eux ainsi qu'avec l'environnement qui les supporte, et par les principes qui guident la conception et l'évolution du logiciel.



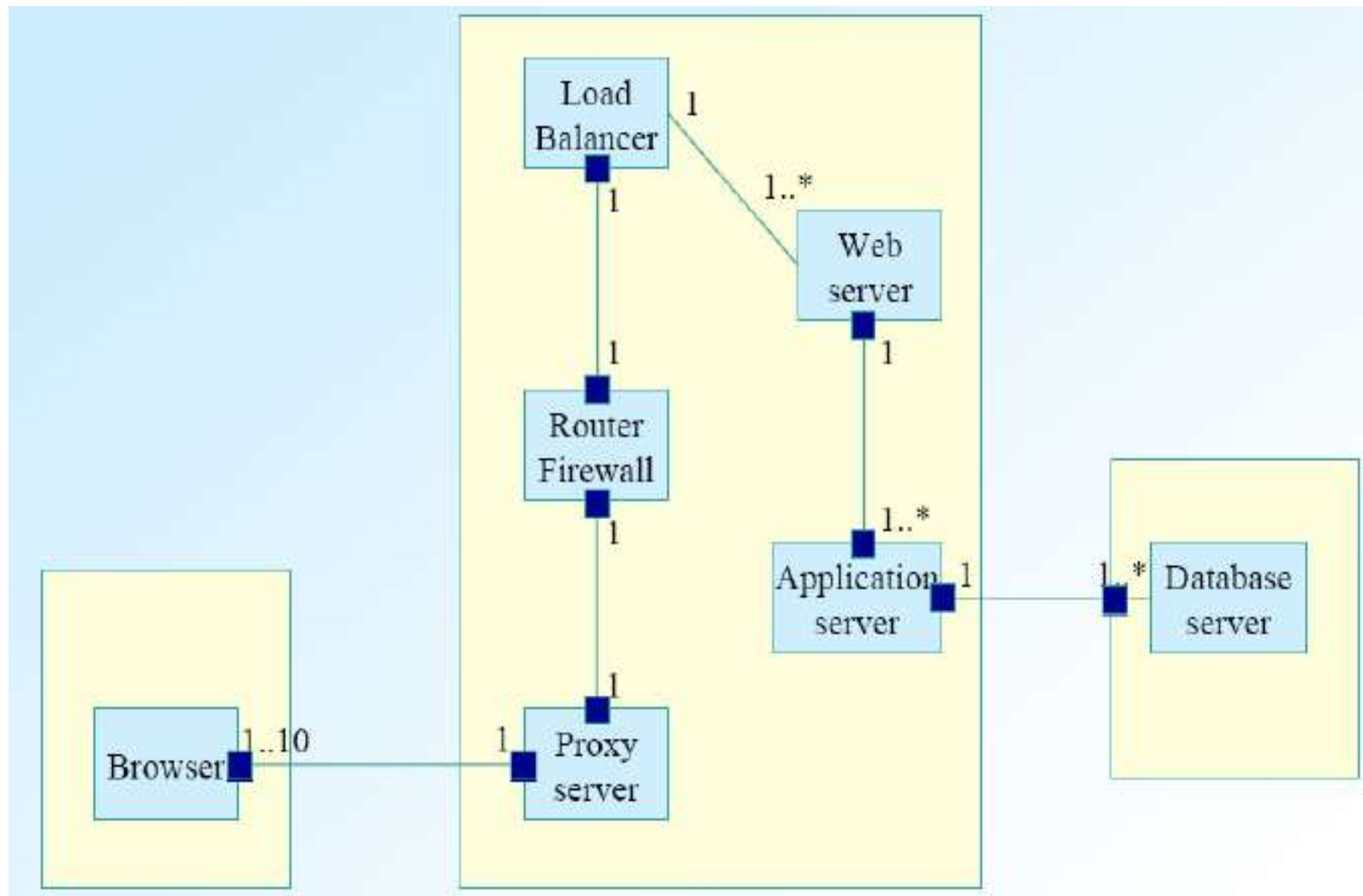
- Introduit les notions et concepts de découpage en couches, modules, composants, design patterns et frameworks.
- Une architecture logicielle est une représentation abstraite d'un système exprimée essentiellement à l'aide de composants logiciels en interaction via des connecteurs.
 - Ne fournit que les propriétés externes des éléments structurants
 - Ne se préoccupe pas des détails d'implantation

- L'architecture informatique définit la structuration d'un système informatique (i.e. matériel et logiciel) en termes de composants et d'organisation de ses fonctions
- La définition de l'architecture logicielle consiste à:
 - Décrire l'organisation générale d'un système et sa décomposition en sous-systèmes ou composants
 - Déterminer les interfaces entre les sous-systèmes
 - Décrire les interactions et le flot de contrôle entre les sous-systèmes
 - Décrire également les composants utilisés pour implanter les fonctionnalités des sous-systèmes
 - Les propriétés de ces composants
 - Leur contenu (e.g., classes, autres composants)
 - Les machines ou dispositifs matériels sur lesquels ces modules seront déployés

EXEMPLE: L'ARCHITECTURE D'UNE APPLICATION D'E-COMMERCE



DÉFINITIONS



- La structuration du système peut être vue sous différents angles, selon que l'on considère :
 - **le découpage « logique »** → Vue Logique
 - montre le découpage en modules sans se soucier des détails physiques d'exécution (machines, OS, réseaux, etc.)
 - **Vue en couches = layer view**
 - **le découpage « physique »** → Vue Physique
 - s'intéresse au déploiement physique prenant en compte le contexte d'exécution.
 - **Vue en niveaux = tier view**
- la littérature propose des modèles standards de structuration qui couvrent les types classiques d'applications (**Styles architecturaux**)

- Pourquoi développer une architecture logicielle ?
 - Pour permettre à tous de mieux comprendre le système
 - Pour permettre aux développeurs de travailler sur des parties individuelles du système en isolation
 - Pour préparer les extensions du système
 - Pour faciliter la réutilisation et la réutilisabilité

- Il n'y a pas une architecture unique permettant de réaliser un système, il y en a plusieurs.
- Le choix de l'architecture implique plusieurs choix dont les **technologies**, les **frameworks**, les **serveurs** à utiliser, etc.
- Le choix dépend des exigences fonctionnelles et non fonctionnelles du logiciel
- Choix favorisant la stabilité : l'ajout de nouveaux éléments sera facile et ne nécessitera en général que des ajustements mineurs à l'architecture
- Influencé par certains « modèles connus » de décomposition mineurs à l'architecture en composants (styles architecturaux)

STYLES ARCHITECTURAUX



Plan

1. **Définitions d'un style architectural**
2. Style architectural Vs Patron de conception
3. Typologie des styles architecturaux
4. Les styles architecturaux logiques
5. Les styles architecturaux physiques

Un style architectural ou Patron d'architecture

- Est un patron décrivant une architecture logicielle permettant de résoudre un problème particulier
- Définit :
 - Un ensemble de composants et de connecteurs (et leurs types)
 - Les règles de configuration des composants et des connecteurs (topologie)
 - Une spécification du comportement du patron
 - Des exemples de systèmes construits selon ce patron
- Constitue un modèle éprouvé et enrichi par l'expérience de plusieurs développeurs (compréhensibilité, maintenance, évolution, réutilisation, performance, documentation, etc).

Définition

- Un style architectural est un *patron récurrent* (commun) d'organisation / architecture de système
- Un style architectural est déterminé par :
 - ❑ Un ensemble de *types* de composantes (e.g. dépôt de données, processus, procédure),
 - ❑ Une structuration *topologique* des composantes indiquant leurs relations durant l'exécution
 - ❑ Un ensemble de *contraintes sémantiques* sur les composantes et leurs interrelations
 - ❑ Un ensemble de *connecteurs* (e.g. appel de procédures, interruptions, etc.) pour médier la communication et la coordination entre les composantes.

Plan

1. Définitions d'un style architectural
2. **Style architectural Vs Patron de conception**
3. Typologie des styles architecturaux
4. Les styles architecturaux logiques
5. Les styles architecturaux physiques

Patron d'architecture - Patron de conception

- **Les patrons d'architecture** (architectural patterns) se situent à un niveau plus élevé d'abstraction.
 - Au niveau de la conception globale
 - Le patron d'architecture apporte des solutions sur la manière de concevoir l'organisation à grande échelle (architecture) d'un logiciel en faisant abstraction des détails. Il comporte des guides de bonnes pratiques et des règles générales qui ne peuvent pas être traduites directement en code source.
- **les patrons de conception** (design patterns) sont plus proches de préoccupations locales liées à l'implémentation du code.
 - Au niveau de la conception détaillée
 - Le patron de conception suggère un arrangement, une manière d'organiser des classes. Il décrit une organisation de classes fréquemment utilisée pour résoudre un problème récurrent. Le patron de conception parle d'instances, de rôles et de collaboration.

Plan

1. Définitions d'un style architectural
2. Style architectural Vs Patron de conception
3. **Typologie des styles architecturaux**
4. Les styles architecturaux logiques
5. Les styles architecturaux physiques

Typologie des styles architecturaux

- La structuration du système peut être vue sous différents angles, selon que l'on considère :
 - La vue ou le découpage « **logique** » : indépendant des considérations physiques, hors de tout contexte d'exécution (machines, OS et réseaux)
 - La vue ou le découpage « **physique** » : prenant en compte le contexte d'exécution.

- La description d'un style architectural fixera:
 - la nature des composants et des connecteurs ;
 - les contraintes sur la façon dont le graphe peut être construit.
- Les questions qui vont diriger cette étude comparée sont les suivantes :
 - Comment peut-on décrire ce style ?
 - Quels sont les invariants/propriétés maintenus par ce style ?
 - Quels sont les exemples d'utilisation de ce style ?
 - Quelles sont ses variantes ?
 - Quels sont les avantages et les inconvénients de ce style ?

Typologie des styles architecturaux

logique

Flux de données

Centrés données

Appels & retours

- Multicouches
- MVC
- Orientés objets
- Programme et routines
- Orientés services

Orientés agents

Physique

1 - tiers

2-tiers

- Client - Serveur
- Pair - à - Pair

N - tiers

Plan

1. Définitions d'un style architectural
2. Style architectural Vs Patron de conception
3. Typologie des styles architecturaux
4. **Les styles architecturaux logiques**
5. Les styles architecturaux physiques

4. Styles logiques

- **flots de données**
 - Pipeline (*pipes and filters*), Batch séquentiel
- **centrés sur les données**
 - Dépôt (*repositories*), Tableau noirs (*blackboards*)
- **Call-and-return**
 - Programme principal et sous routines, en couches (*layered*), MVC, Orientés objets, Orientés services.

- Convient aux systèmes de traitement et de transformation de données
 - **Composant** = filtre= composants qui *traitent* l'information.
 - Reçoit ses données d'un ou plusieurs canaux d'entrée.
 - Effectue la transformation/traitement des données et envoie les données de sortie produites sur un ou plusieurs canaux de sorties.
 - Fonctionnent en concurrence: Chacun traite les données au fur et mesure qu'il les reçoit
 - **Connecteur** = Pipe= canaux par lesquels *transitent* l'information.
 - Unidirectionnel au travers duquel circulent un flot de données (stream).
 - Synchronisation et utilisation d'un tampon parfois nécessaire pour assurer le bon fonctionnement entre filtre producteur et filtre consommateur
- La **configuration** détermine l'ordre des traitements

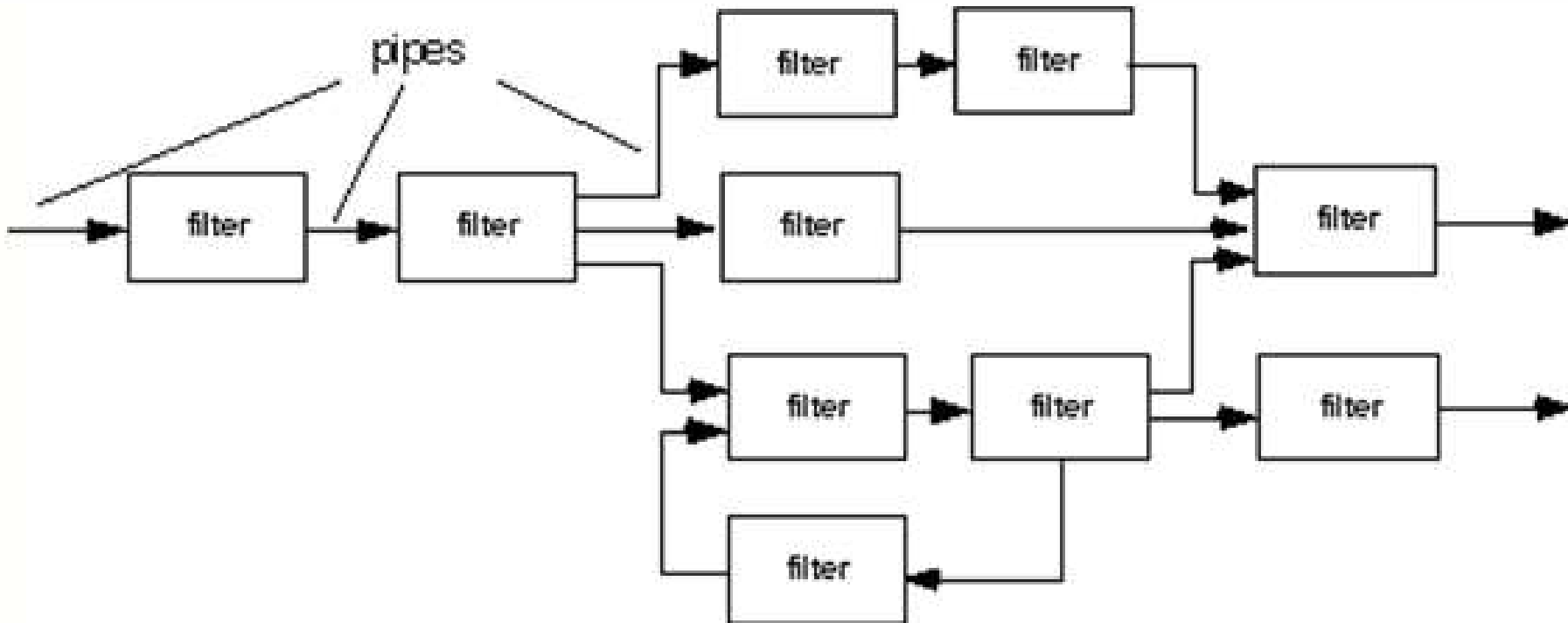
4.1 Styles de flots de données

Pipeline (*pipes & filters*) : Architecture composée de «filtres» connectés par des «canaux de transfert».

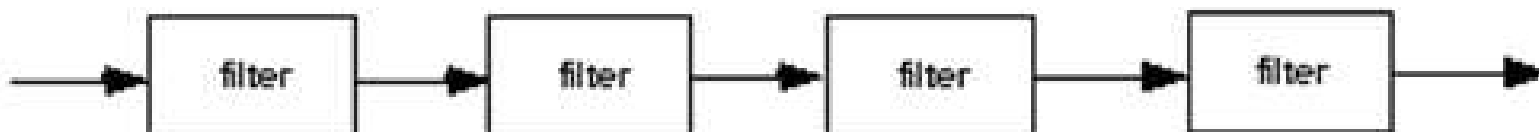
- **Filtre** = composant de transformation indépendant. C'est un convertisseur qui attend des données d'entrée et produit des données de sortie dans des formats (types) prédéfinis.
- **Pipes ou canaux** : connecteurs qui relient les composantes source et récepteur et propagent les données. Échange de données entre les filtres

Batch : une variante de l'architecture pipeline où les filtres sont en séquence.

4.1 Styles de flots de données



(a) pipes and filters



(b) batch sequential

□ Architecture comprenant:

Filtres : composantes qui traitent un *stream* de données. Sont des transformations qui convertissent leur *inputs* en *outputs*

Pipelines : connecteurs qui relient les composantes *source* et *récepteur* et propagent les données tel un *stream*

□ Inspiré des *shells* de Unix

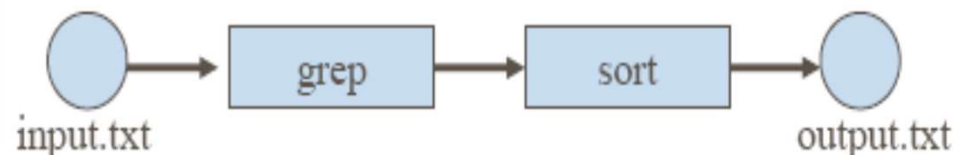
□ `cat input.txt | grep "text" | sort > output.txt`

□ Modèle strictement dirigé par les données

□ Échange de données entre les filtres

□ Les filtres ne partagent pas d'états entre eux

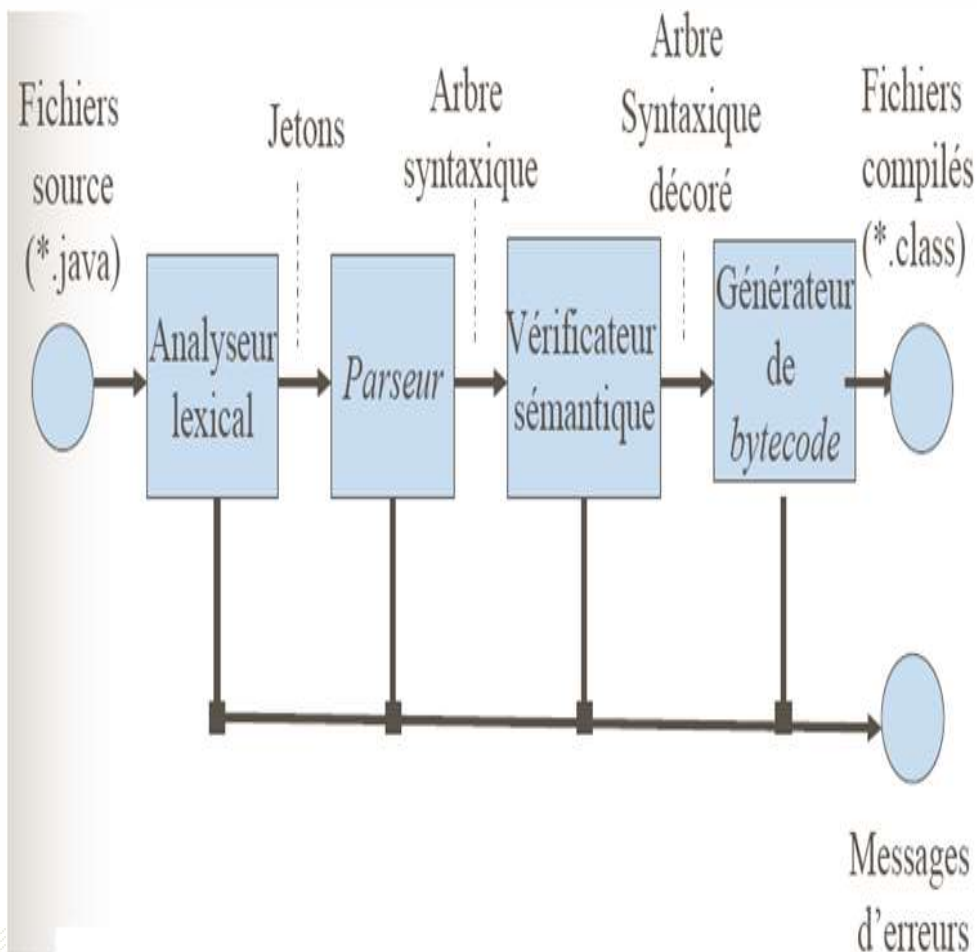
□ Les filtres sont spécifiés uniquement en fonction des *inputs* qu'ils acceptent et des *outputs* qu'ils produisent (aucune interaction)



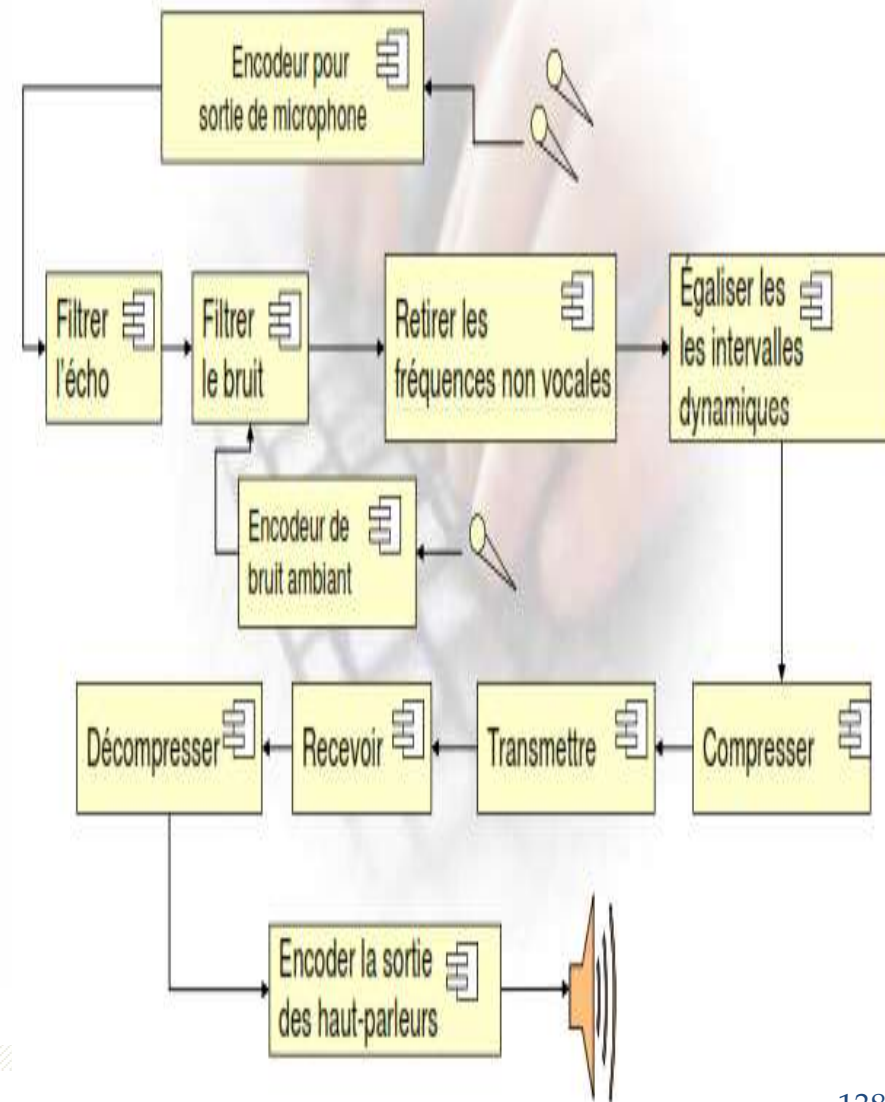
STYLE D'ARCHITECTURE CENTRÉ FLOTS DE DONNÉES

Section 4 Les styles logiques

- Exemples : application de traitement de son, Compilateur (analyse lexicale, syntaxique, sémantique)
 - Architecture de compilateur



Système de traitement du son



Avantages

- **Diviser pour régner** : les filtres peuvent être conçus séparément
- **Cohésion forte** : les filtres sont un type de cohésion fonctionnelle
- **Couplage faible** : les filtres n'ont qu'une entrée et une sortie en général
- **Abstraction** : les filtres cachent généralement bien leurs détails internes
- **Réutilisabilité et réutilisation** : les filtres peuvent très souvent être réutilisés dans d'autres contextes

Inconvénients

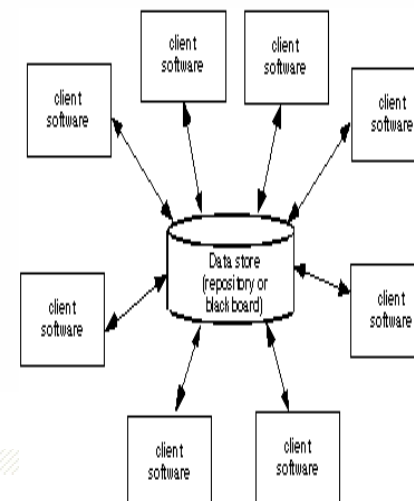
- Les performances dépendent des pipes
- Difficulté de supporter les systèmes interactifs (avec événements)

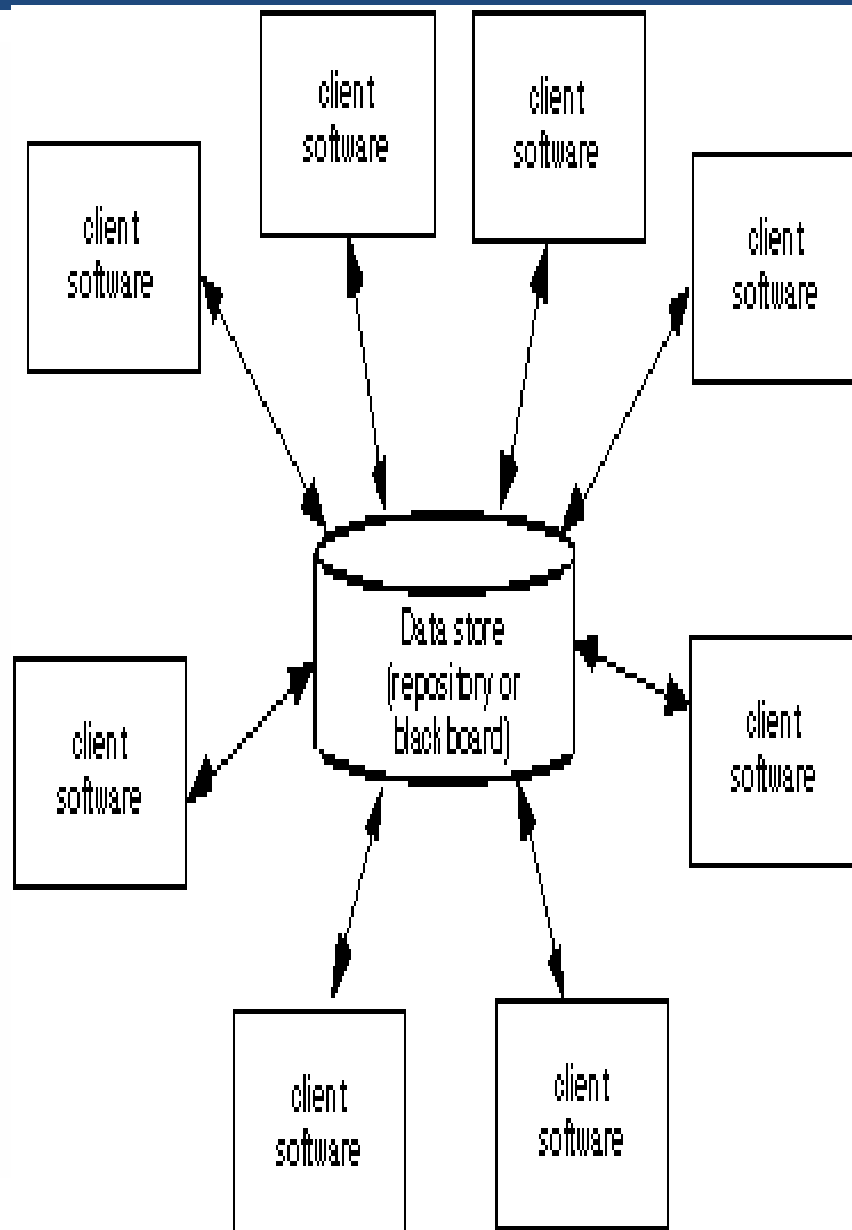
4.2 Styles centrés sur les données

- Dans cette architecture, un composant central (SGBD, Datawarehouse, Blackboard) est responsable de la gestion des données (conservation, ajout, retrait, mise-à-jour, synchronisation, ...) .
- Les composants périphériques, baptisés clients ou accesseurs, utilisent le composant central, baptisé serveur de données (centre de données).
- Centre de données
 - Peut être **passif (dépôt de données)**
 - Peut être **actif (tableau noir)** le centre de données constitue alors une structure logicielle où une mémoire de travail appelée «tableau noir», accessible à plusieurs composants-clients, qui sert à communiquer des résultats intermédiaires et à coordonner ces transferts d'informations entre clients.

- Exemples :

- Système bancaire
- Système de facturation
- Environnement de programmation





- Composants: Un composant central détenant les données et des processus indépendants.
- Connecteurs : Des transactions.
- Si c'est l'état du composant central qui provoque l'exécution des processus, on parle l'architecture à tableau noir ("blackboard architecture").
- Si ce sont les modifications apportées au composant central qui provoque l'exécution des processus, il s'agit d'une base de données classique.

- Parfois, les composantes d'un système doivent partager des données
- Options :
 - Conserver les données dans un dépôt centrale (BD ou fichiers) auquel peuvent accéder tous les sous-systèmes.
 - Maintenir dans chaque sous-système un dépôt (par ex. une BD) et passer explicitement les données d'un sous-système à un autre.
- Lorsqu'on a un grand volume de données à partager, un *dépôt commun* est habituellement utilisé
- Exemple : environnement de programmation : Architecture d'un outil CASE (Computer Aided Software engineering)

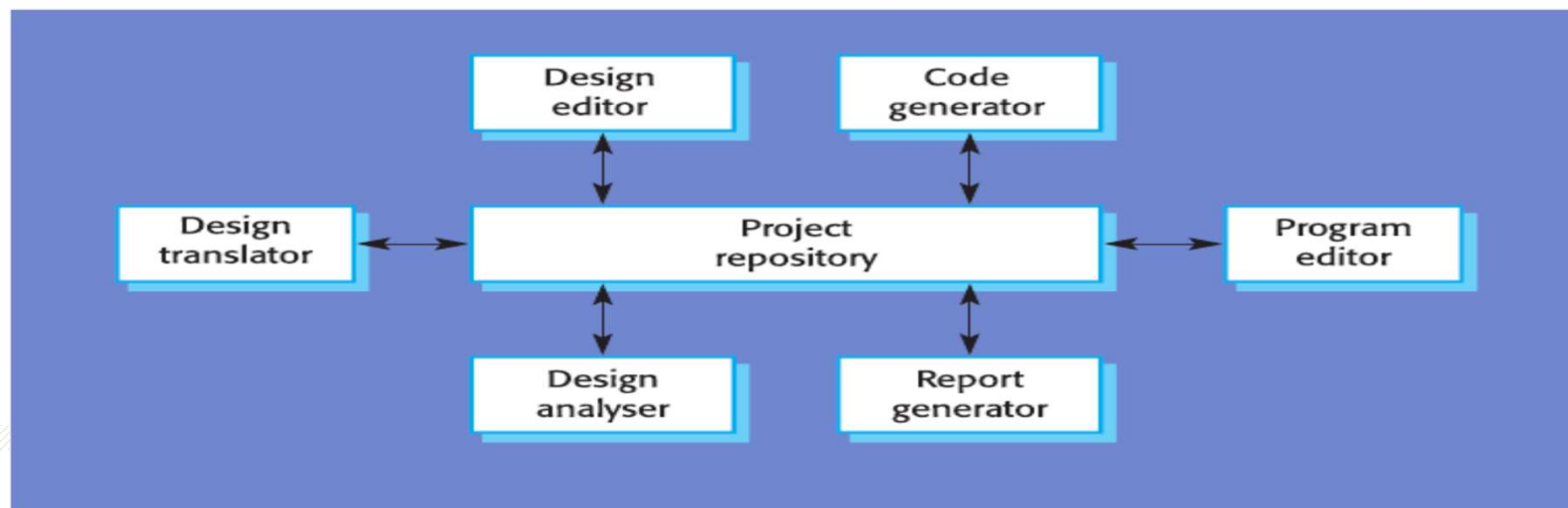
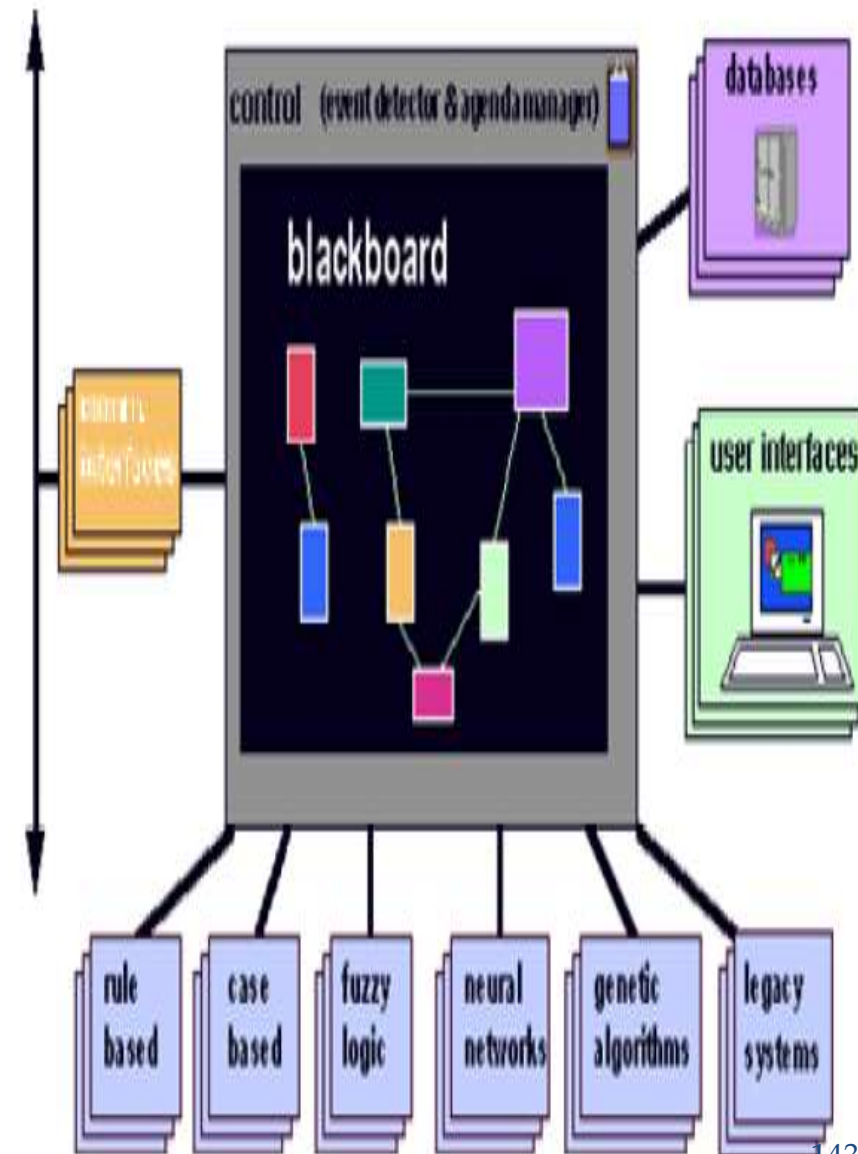
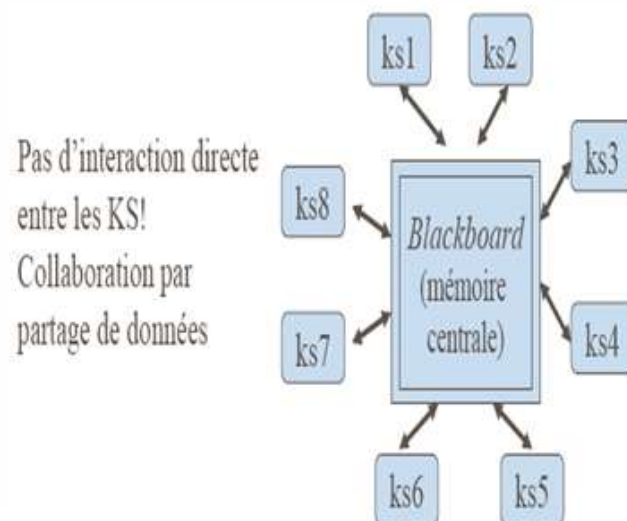


TABLEAU NOIR (*BLACKBOARD*)

Section 4 Les styles logiques

□ Système qui contient:

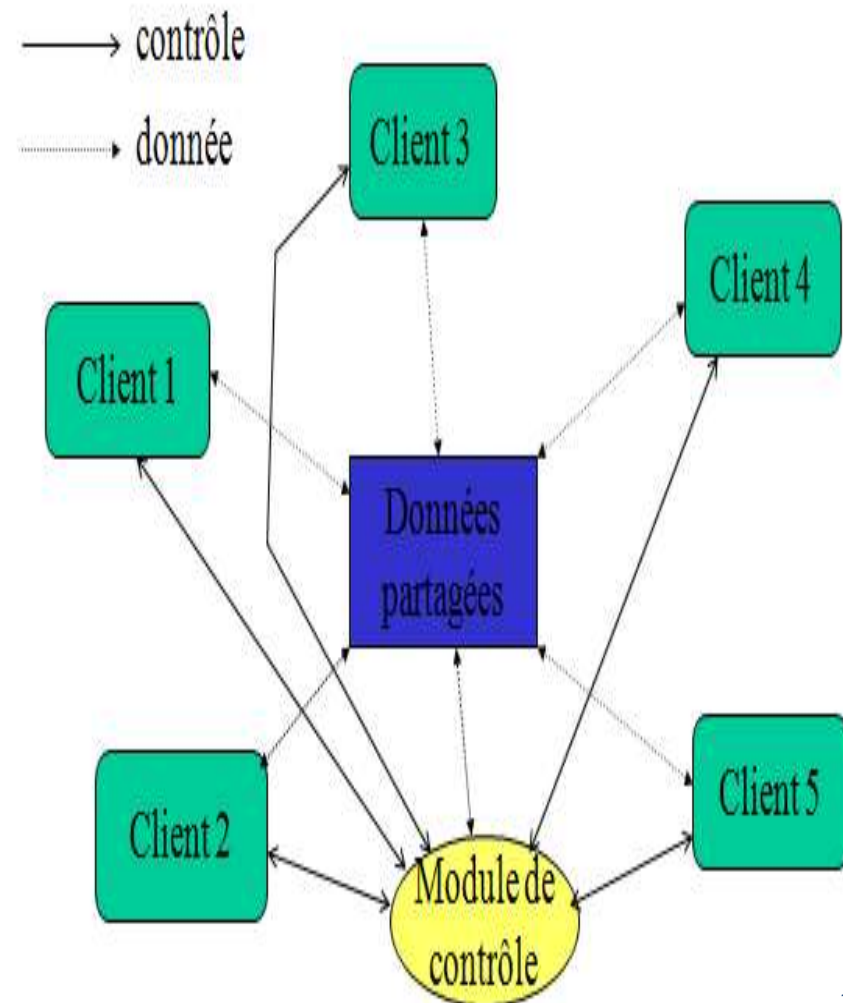
- un dépôt central de donnée– la structure de tableau noir (*blackboard*)
- des sources de connaissance (KS) – les composantes de traitement qui dépendent de l'application



En pratique :

- ❑ Besoin d'un module de contrôle élaboré
- ❑ Associer les événements du *blackboard* aux KS
- ❑ Choix des prochains KS qui modifient le contenu du tableau

Architecture blackboard



4.2 Styles centrés sur les données

Avantages

- Manière efficace pour partager de grand volume de données
- Avantageux pour les applications impliquant des tâches complexes et nombreuses sur les données. Les sous-systèmes n'ont pas à se préoccuper comment les données sont stockées et gérées (*backup*, contrôle d'accès, sécurité...)

Inconvénients

- Le dépôt de données peut facilement constituer un goulot d'étranglement, tant du point de vue de sa performance que du changement (trop d'accès simultanés)

4.3 Styles appels et retours

- Le style « **call and return** » est simple et classique décomposant le système en une hiérarchie de contrôle.
- Ce style, également appelé style **requête-réponse** ou aussi décomposition fonctionnelle, consiste à découper une fonctionnalité en sous-fonctionnalités qui sont également divisées en sous sous-fonctionnalités et ainsi de suite;
- Si à l'origine cette architecture était fondée sur l'utilisation de fonctions, le passage à une méthode modulaire ou objet est tout naturel. Le terme hiérarchie de contrôle est alors utilisé pour décrire l'extension de cette architecture au paradigme modulaire ou objet. Une forme dérivée de cette architecture est l'architecture distribuée où les fonctions, modules ou classes se retrouvent répartis sur un réseau.

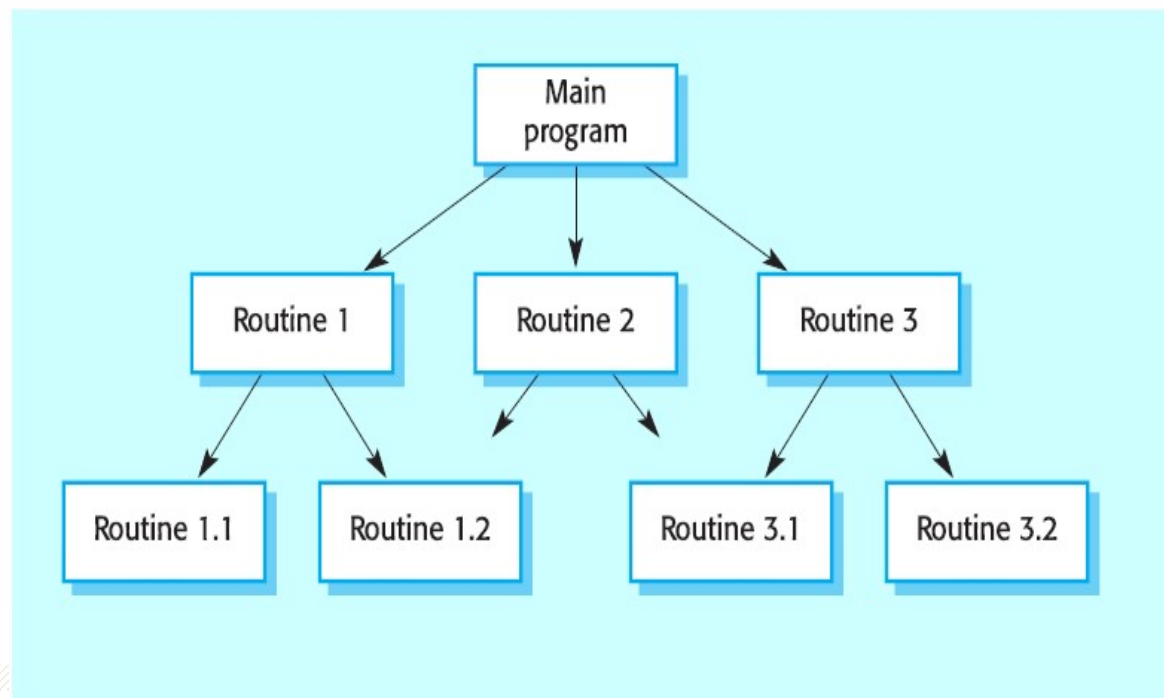
Plusieurs Souches:

- Programme principal et sous-routines
- Orientée Objets
- Orientée Services
- En couches (layered)
-

4.3.1 Programme principal-sous-routines

Modèle Programme principal et sous-routines

- Approche de contrôle centralisée
- Contrôle descendant (top-down)
- **Composantes** : les sous-routines
- **Connecteurs** : les appels fonctionnels

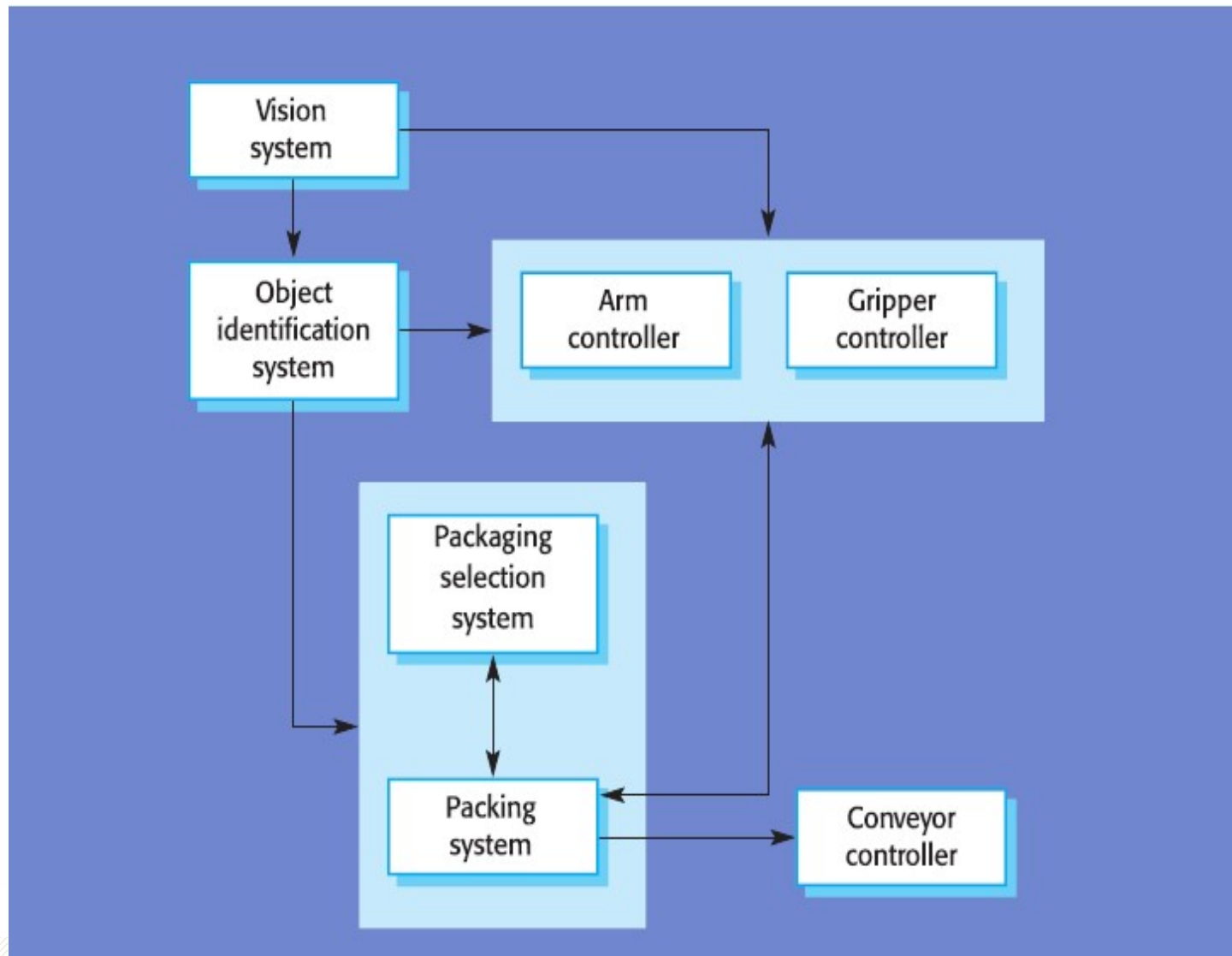


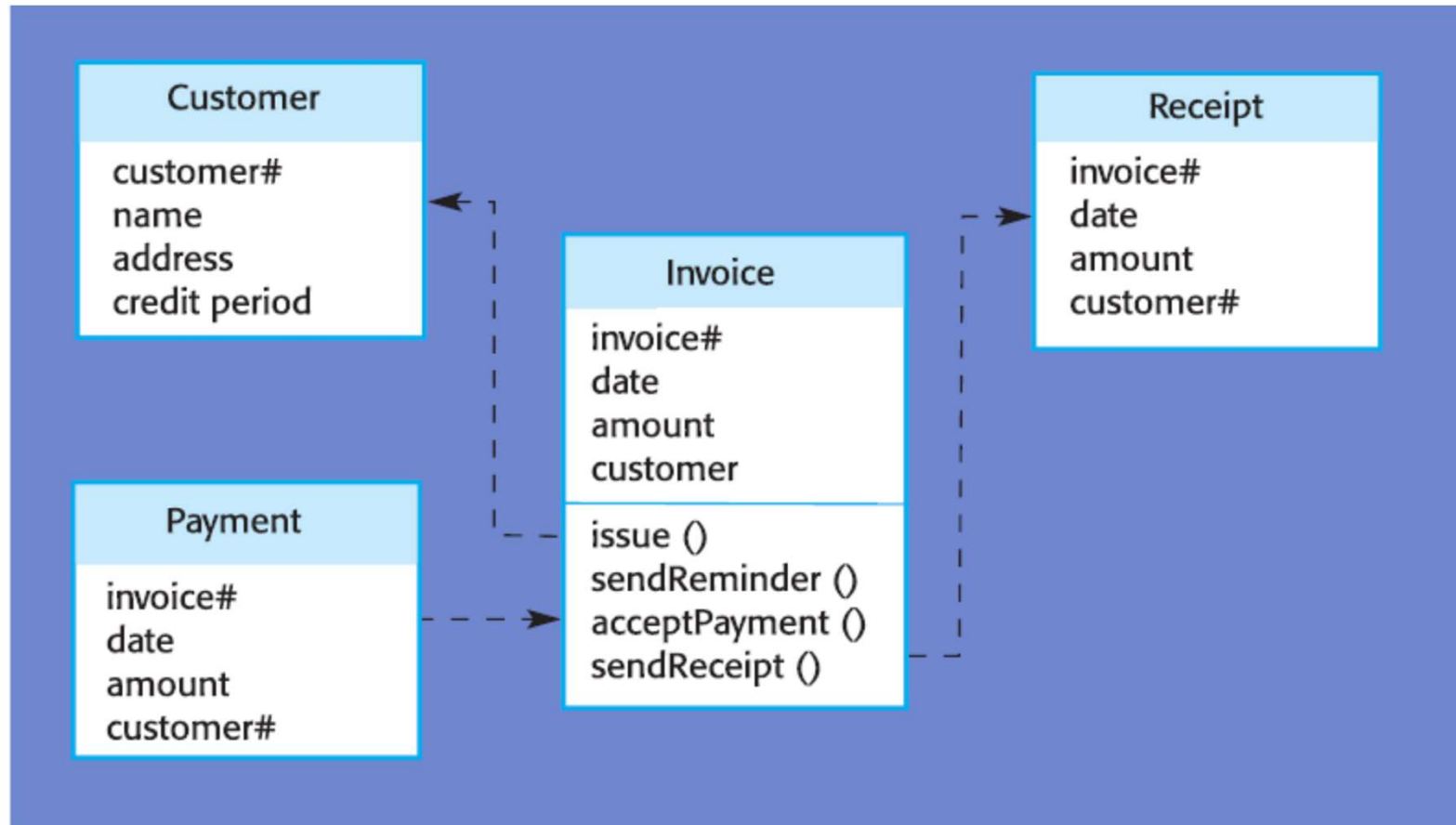
4.3.2 Architecture orientée objets

- Architecture orientée objets se base sur la structuration d'un système en un ensemble de modules faiblement couplés
 - Composantes : paquets ou composants
 - Connecteurs : les interfaces bien définies des composants
- Pour l'implantation, on ajoute un modèle de contrôle
 - Diagramme de séquence ou de collaboration
 - Permet de coordonner les opérations des objets

EXEMPLE : SYSTÈME DE CONTRÔLE D'EMBALLAGE

Section 4 Les styles logiques





■ Avantages

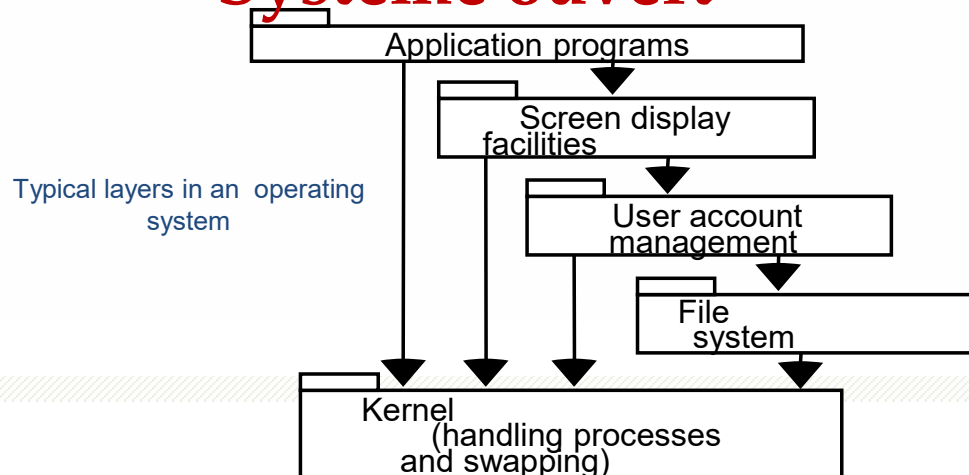
- Faible couplage entre les objets, ce qui rend possible leur modification sans affecter les autres objets.
- Les objets représentent des entités du domaine d'application.
- Tout le monde programme en orienté-objet!
- Représente les fonctionnalités verticales du système => aspects métiers.

■ Inconvénients

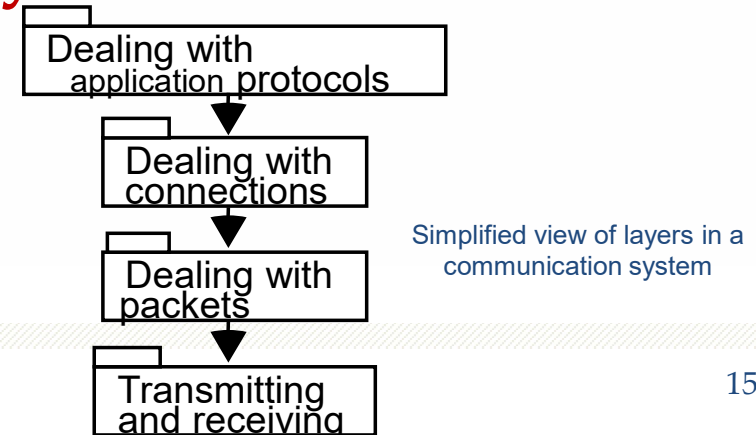
- Les entités complexes sont parfois difficiles à représenter comme des objets.
- Les fonctionnalités horizontales ou traverses => aspects techniques de l'application → s'étalent sur un grand nombre de modules.
- Éléments de code impossible à centraliser, à factoriser (=> mettre les instructions à factoriser en une même méthode)
- Héritage ne permet pas d'éviter complètement la duplication du code,

- **Composants** : les couches
 - Chaque couche réalise un service
- **Connecteurs** : dépendent du protocole d'interaction souhaité entre couches
 - **Système fermé** : une couche n'a accès qu'aux couches adjacentes. Les connecteurs ne relient que les couches adjacentes
 - **Système ouvert** : toutes les couches ont accès à toutes les autres. Les connecteurs peuvent relier deux couches quelconques

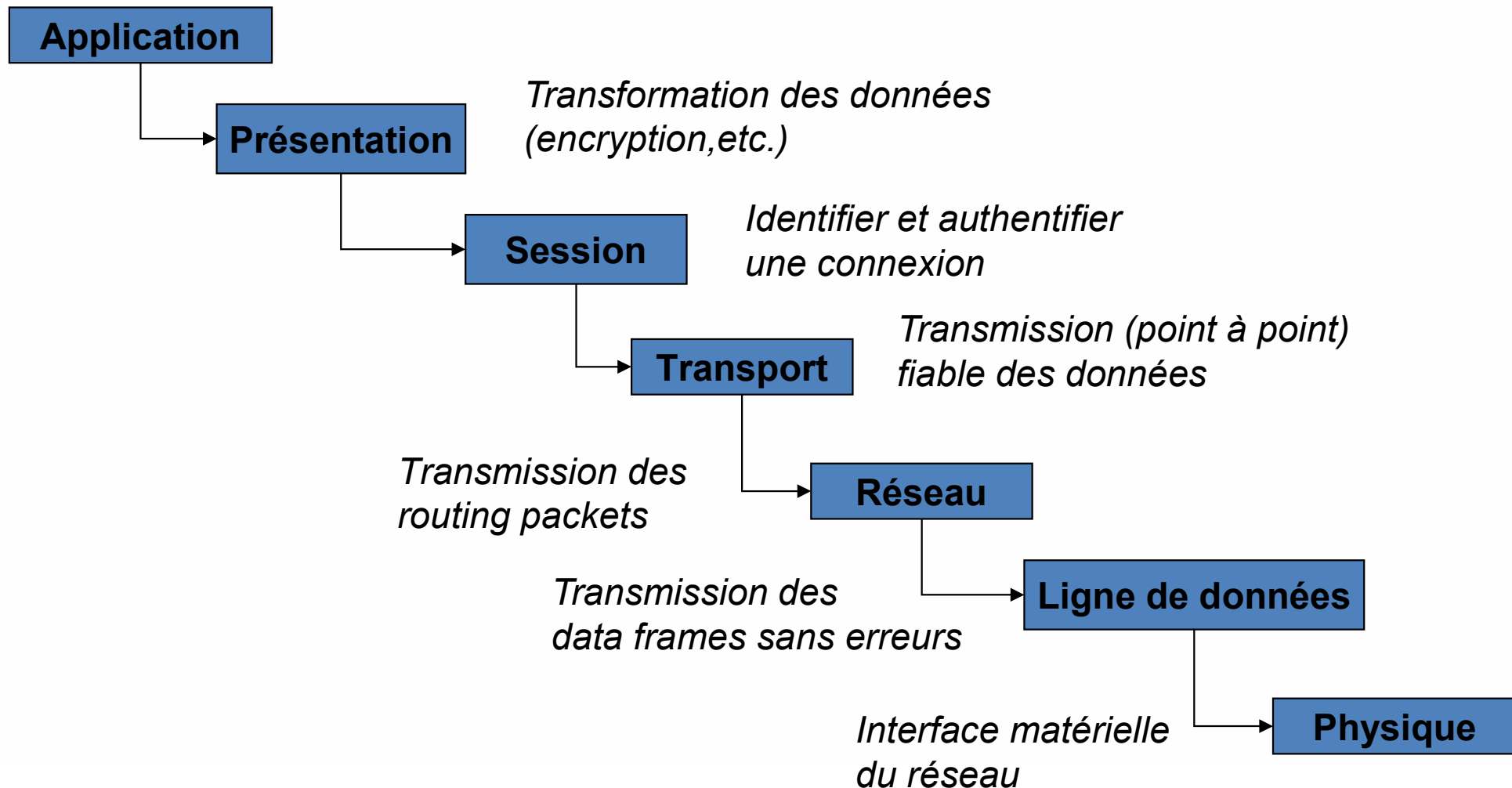
Système ouvert



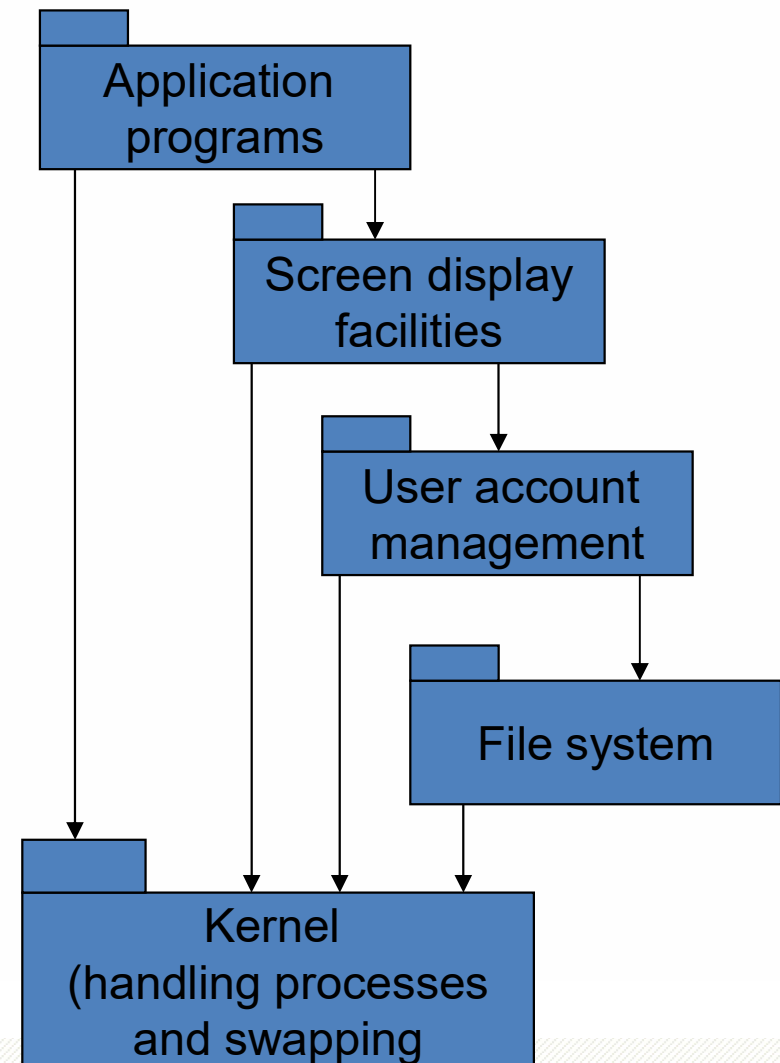
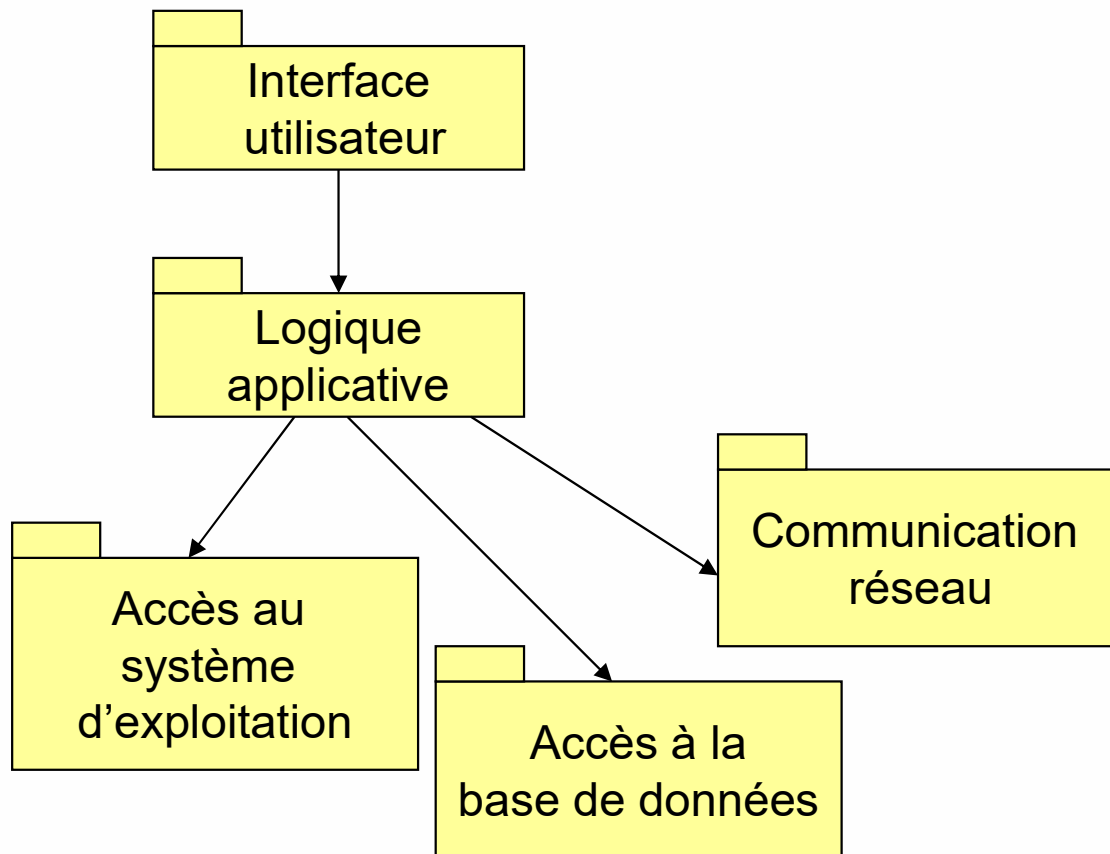
Système fermé



■ Exemple Système fermé: Reference Model of Open Systems Interconnection (OSI model)



■ Exemples système ouvert



- Chaque couche est un composant avec une interface bien définie utilisée par la couche juste au-dessus (i.e., externe)
- La couche supérieure (externe) voit la couche inférieure (interne) comme un ensemble de services offerts
- Il est important d'avoir une couche séparée pour l'IU
 - Les couches juste au-dessous de l'IU offrent les fonctions applicatives définies par les cas d'utilisation
- Les couches les plus basses offrent les services généraux E.g., communication réseau, accès à la base de données

Avantages

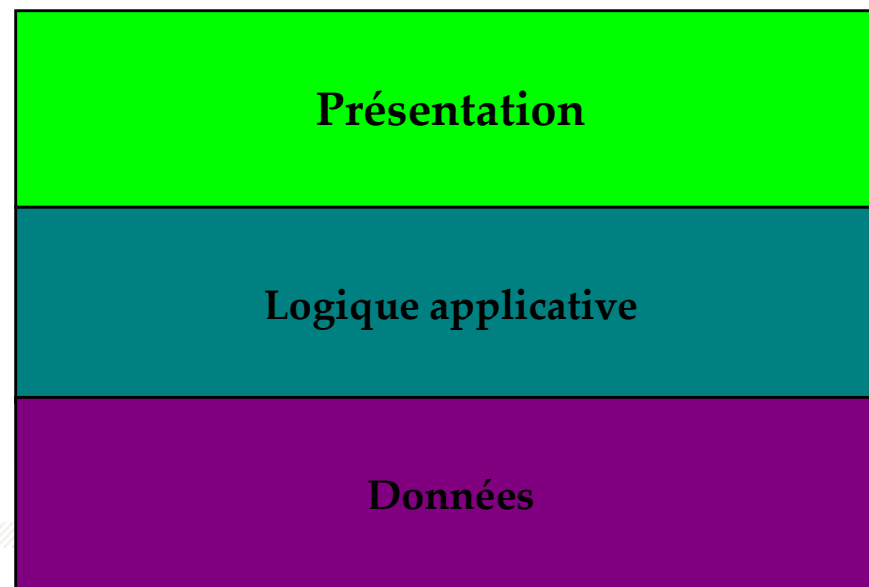
- Diviser pour régner : les couches peuvent être conçues séparément
- Cohésion : des couches bien conçues seront cohésives
- Couplage : des couches inférieures bien conçues ne devraient rien savoir à propos des couches supérieures
- Abstraction : on n'a pas à connaître les détails d'implémentation des couches inférieures
- Réutilisabilité et Réutilisation

Inconvénients

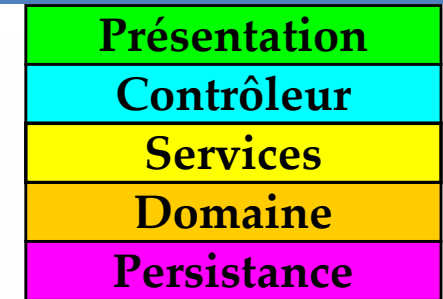
- La bonne décomposition en couches

Variante 1: Modèle trois couches

- **La couche présentation ou IHM** (Interface Homme/Machine)
 - gère les interactions utilisateur/machine, la présentation
- **La couche Logique applicative**
 - contrôles effectués au niveau du dialogue avec l'IHM
 - Services métiers de l'application
- **La couche données**
 - Gère le stockage des données et l'accès à ces dernières



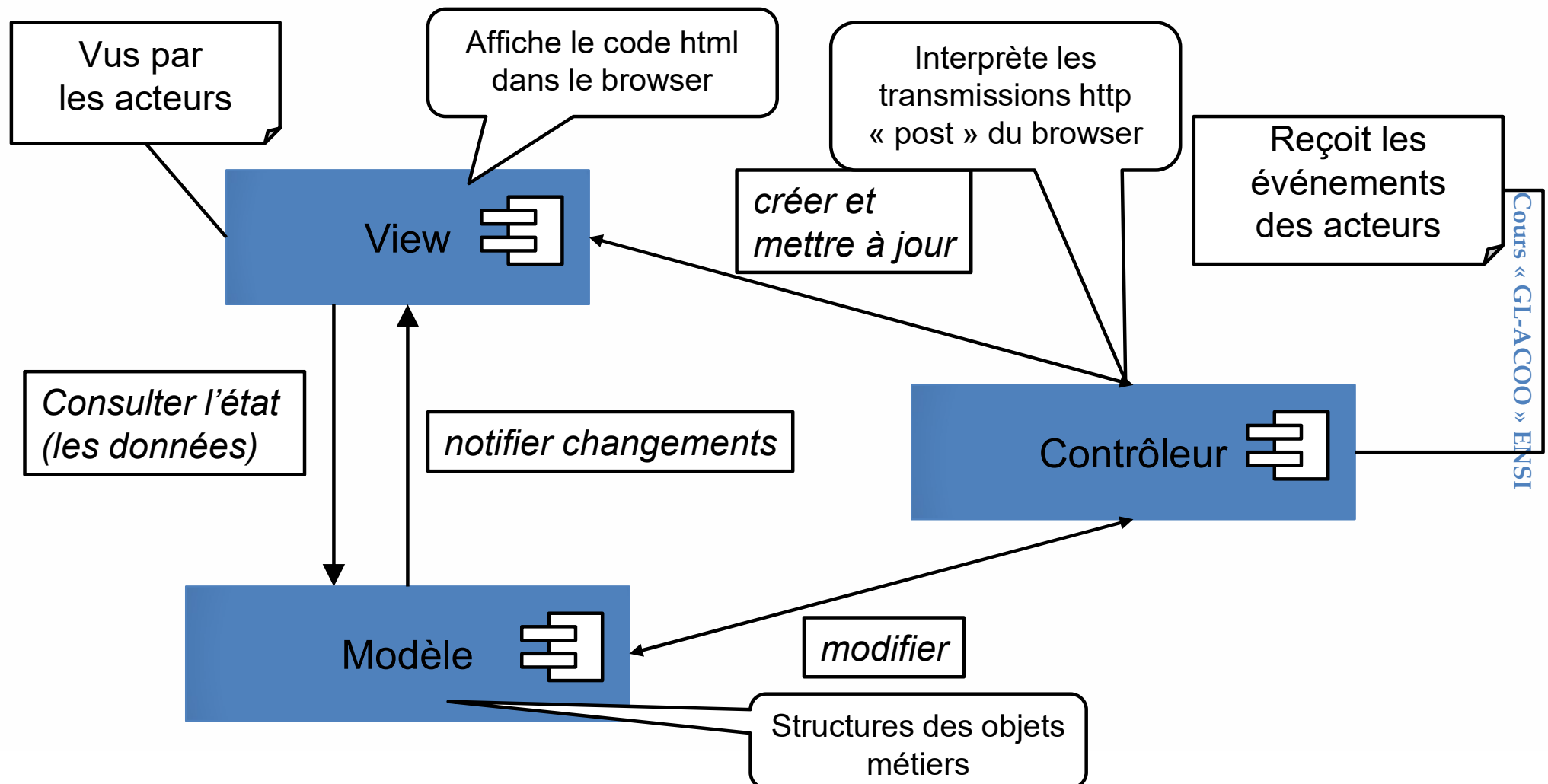
Variante 2: Modèle cinq couches



- **Présentation** : assure l'affichage des IHM
- **Contrôleur/Coordination** : le contrôle de la cinématique des écrans et l'invocation des appels de services
- **Service** : implémente la logique métier
- **Domaine** : la gestion des objets métiers
 - Exemple : l'opération de virement de compte à compte
 - l'opération de virement est un élément de la couche Services
 - le compte bancaire et le client se situent dans la couche Domaine
- **Persistance** : fournit des services de stockage des données et de mapping entre les différentes formes des objets métiers du domaine (exemple : hibernate permet le mapping objet-relationnel).
- Chaque couche a ses propres responsabilités et utilise la couche située en dessous d'elle
- En fonction du projet, les architectes enrichissent et élaguent le modèle. La structuration est alors guidée par les contraintes exprimées et existantes

- Une autre variante : le MVC
- L'architecture applicative de gestion des interactions utilisateur est généralement mise en œuvre autour du motif de conception MVC (Modèle-Vue-Contrôleur):
 - le **Modèle** représente l'ensemble des composants qui sont chargés de réaliser des appels à la couche Services et de mettre les résultats de l'appel à la disposition de la Vue
 - la **Vue** représente l'interface utilisateur.
 - le **Contrôleur** gère la synchronisation entre la Vue et le Modèle. Le contrôleur réagit aux actions de l'utilisateur en effectuant les actions nécessaires sur le Modèle. Le Contrôleur surveille les modifications du modèle et informe la Vue des mises à jour nécessaires.

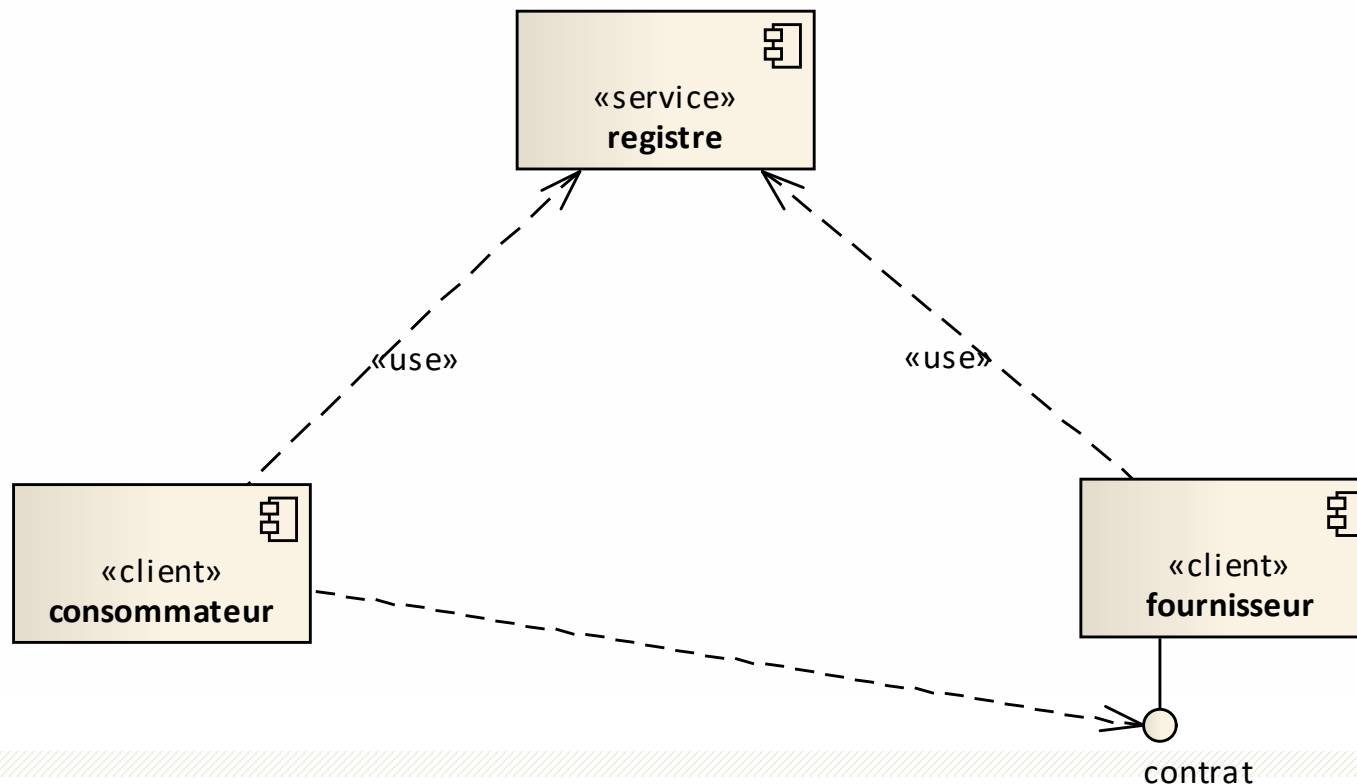
4.3.4 Architecture multi-couches



- **Avantages** : approprié pour les systèmes interactifs, particulièrement ceux impliquant plusieurs vues du même modèle de données. Peut être utilisé pour faciliter la maintenance de la cohérence entre les données distribuées
- **Inconvénient** : goulot d'étranglement possible
- D'un point de vue conception
 - **Diviser pour régner** : les composants peuvent être conçus indépendamment
 - **Cohésion** : meilleure cohésion que si les couches vue et contrôle étaient dans l'interface utilisateur.
 - **Couplage** : le nombre de canaux de communication entre les 3 composants est minimal
 - **Réutilisabilité** : la vue et le contrôle peuvent être conçus à partir de composants déjà existants
 - **Flexibilité** : il est facile de changer l'interface utilisateur
 - **Testabilité** : il est possible de tester l'application indépendamment de l'interface

- Type d'architecture reposant sur les standards de l'Internet
- permettant à des applications de communiquer sans préoccupation des technologies d'implantations utilisées de part et d'autre
- Priorité : **Interopérabilité**
- SOA est basée sur des services **faiblement couplés**, **indépendants** des protocoles, basés sur les **standards** et **distribués**.
- Une architecture SOA est basé sur des **services** :
 - Qui sont **autonomes**
 - Qui sont **composables** : créer un service à partir d'autres services
 - Qui sont **réutilisables**
 - Qui sont **basés sur des standards**
- Un service SOA dialogue avec ses consommateurs sous une forme **standardisée**, tant sur le plan technique que sur le plan métier

- Les composants de SOA:
 - Le **consommateur** utilise le service
 - Le **fournisseur** assure le service
 - Le **registre** fait le lien entre le fournisseur et le consommateur

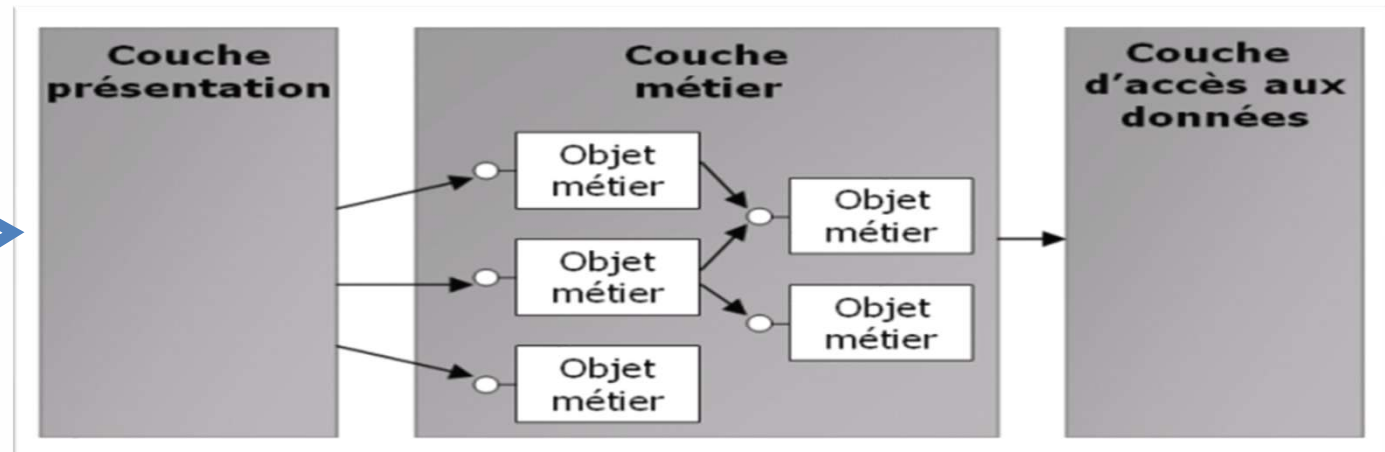


4.5 Architecture orientée services

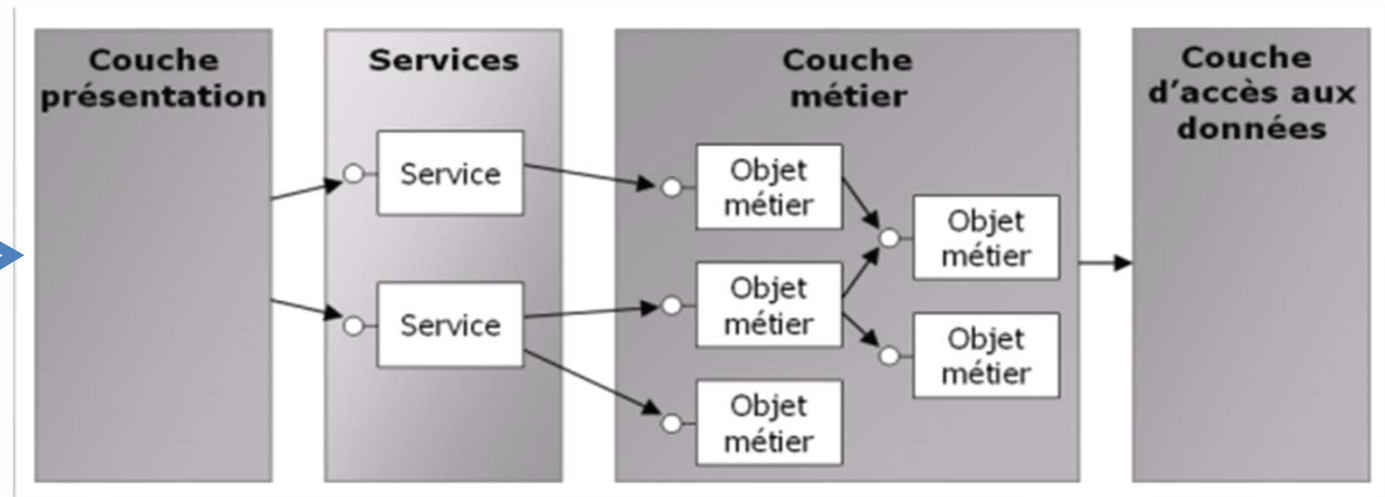
- L'architecture orientée service constitue un **style d'architecture** basée sur le principe de séparation de l'activité métier en une série **de services**. »
- « Ces services peuvent **être assemblés et liés** entre eux selon le principe de couplage lâche pour exécuter l'application désirée. »
- « Ces services sont définis à un niveau supérieur de la traditionnelle approche **composants** »
- L'AOS se base sur l'émergence d'une couche de services. Ces services offrent une vue logique des traitements et données existant déjà ou à développer.
- Un service, fourni par un **fournisseur**, met à la disposition de ses **consommateurs** (humains ou logiciels) intervenants dans des processus métiers, un accès vers une ou plusieurs fonctions métiers.
- Un service SOA dialogue avec ses consommateurs sous une forme **standardisée**, tant sur le plan technique que sur le plan métier

4.5 Architecture orientée services

AOO



Modèle (SOA) AOS



Services ?

■ Modèle orienté objets (POO)

Le code client doit jongler directement avec le modèle objet de la couche métier, ce qui a pour conséquence de lier celle-ci très fortement à un modèle spécifique et requiert un nombre d'appels important entre les deux couches. La multiplication des appels entre couches pose problème lors de la mise à disposition à distance des objets métiers, et que le nombre d'objets à manipuler réduit l'indépendance entre couches et complexifie la prise en main de la couche métier.

■ Modèle orienté services (SOA):

On remarque que l'on a introduit un niveau d'indirection supplémentaire sous la forme de *services*. La couche présentation ne manipule plus directement les objets métiers, mais passe par des services. Les objets métiers se trouvent dans des bibliothèques de classes directement chargées dans le même processus que les services et par conséquent coût des appels aux objets métiers est très faible.

Les services agissent comme des boîtes noires faisant abstraction de la complexité du modèle objet, présentant un ensemble de fonctionnalités restreints et permettant de réduire les échanges entre les couches.

4.5 Architecture orientée services

■ Avantages

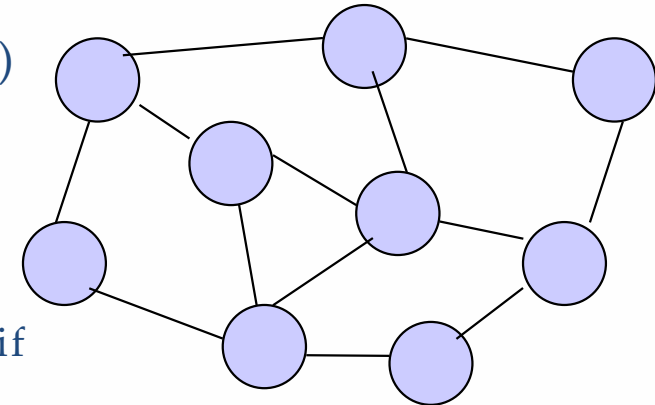
- Interopérabilité
- Indépendance et facilité de découverte
- Réutilisation/Réutilisabilité
- Permet l'utilisation des applications depuis n'importe quel équipement (PC, mobile, etc...)
- Localisation et interfaçage transparents (Les services agissent comme des boîtes noires)
- Possibilité de mise en place facilitée à partir d'une application objet existante
- Réduction des coûts en phase de maintenance et d'évolution car les services sont par construction autonomes
- Facilité d'amélioration des performances pour des applications importantes (répartition des traitements facilitée)

■ Désavantage

- Nécessité d'appréhender de nouvelles technologies (coûts)

- L'architecture orientée agents correspond à un paradigme où l'objet, de composant passif, devient un composant projectif :
- En effet, dans la conception objet, l'objet est essentiellement un composant passif, offrant des services, et utilisant d'autres objets pour réaliser ses fonctionnalités; l'architecture objet n'est donc qu'une extension de l'architecture en appels et retours, le programme peut être écrit de manière à demeurer déterministe et prédictible.
- L'agent logiciel, par contre, utilise de manière relativement autonome, avec une capacité d'exécution propre, les autres agents pour réaliser ses objectifs : il établit des dialogues avec les autres agents, il négocie et échange de l'information, décide à chaque instant avec quels agents communiquer en fonction de ses besoins immédiats et des disponibilités des autres agents.

- Modèles basés sur des agents
- Système interactif = ensemble d'unités computationnelles (agents)
- Agent
 - capacité à réagir et à gérer des événements
 - caractérisé par un état
 - possède une capacité d'expertise (rôle)
 - relation utilisateur $\Rightarrow$ interacteur / objet d'interactif
 - Entité qui a un état et une expertise et est capable d'émettre et de réagir à des événements
- Correspondance avec l'approche à objets
 - catégorie d'agents (réactifs) $\rightarrow$ classe
 - événement $\rightarrow$ méthode
 - encapsulation : l'agent (l'objet) est seul à modifier directement son état
 - mécanisme de sous-classe $\rightarrow$ modifiabilité
- Système interactif = agents réactifs ($\leftrightarrow$ agents cognitifs)
- Modularité et parallélisme
 - conception itérative (modifiabilité)
 - dialogue à plusieurs fils
 - mise en oeuvre des collecticiels

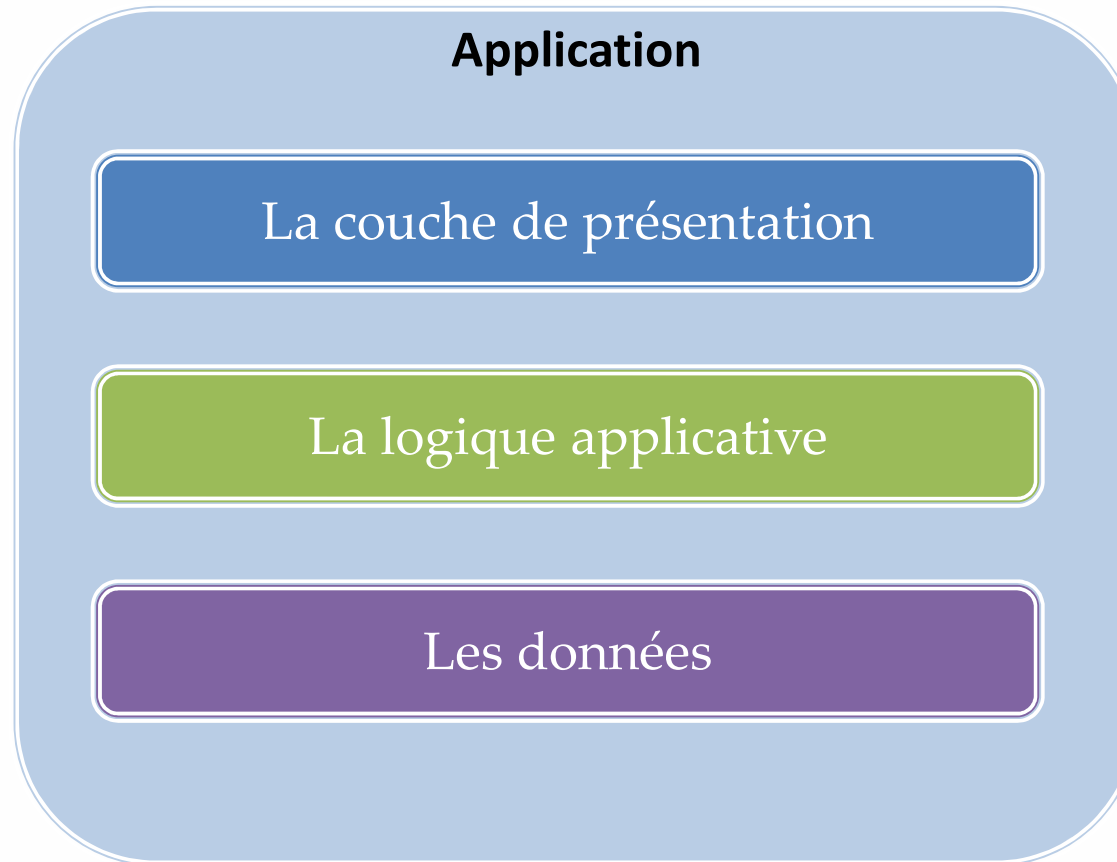


Plan

1. Définitions d'un style architectural
2. Style architectural Vs Patron de conception
3. Typologie des styles architecturaux
4. Les styles architecturaux logiques
5. **Les styles architecturaux physiques**

Les styles physiques

Niveau d'abstraction d'une application



Les styles physiques

En règle générale, une application est découpée en 3 niveaux d'abstraction :

- La couche **présentation** ou IHM (Interface Homme/Machine) qui gère les interactions utilisateur/machine ou la présentation
- La couche **traitements** : qui effectue les services et gère la logique métier. Elle permet de décrire les travaux à réaliser par l'application. Les traitements peuvent être découpés en deux familles :
 - les traitements locaux: Ils regroupent les contrôles effectués au niveau du dialogue avec l'IHM, visant essentiellement le contrôle et l'aide à la saisie
 - les traitements globaux: Ils constituent l'application elle-même.
- La couche **données** qui gère le stockage des données et l'accès à ces dernières (DAL : Data Access Layer): Plus exactement la couche accès aux données. Elle regroupe l'ensemble des mécanismes permettant la gestion des informations stockées par l'application

Et toute application possède ces trois parties : on parle de niveaux logiques qui vont être répartis sur des niveaux physiques ou des tiers (des étages).

Les styles physiques

Ces 3 niveaux peuvent être imbriqués ou répartis de différentes manières entre plusieurs machines physiques suivant les contraintes d'utilisation ou les contraintes techniques :

- Architecture 1-tiers
- Architecture 2-tiers
- Architecture 3-tiers
- Architecture N-tiers

5.1 Architecture 1-tiers

1. Architecture 1-tiers ou Architecture centralisée : tout est sur la même machine

Les trois couches applicatives sont intimement liées et s'exécutent sur le même ordinateur

- Traditionnellement une application informatique est un programme exécutable sur une seule machine qui représente la logique de traitement des données manipulées par l'application.
- Dans ce schéma, les traitements, les données d'entrées, les données de sortie sont sur une seule machine.

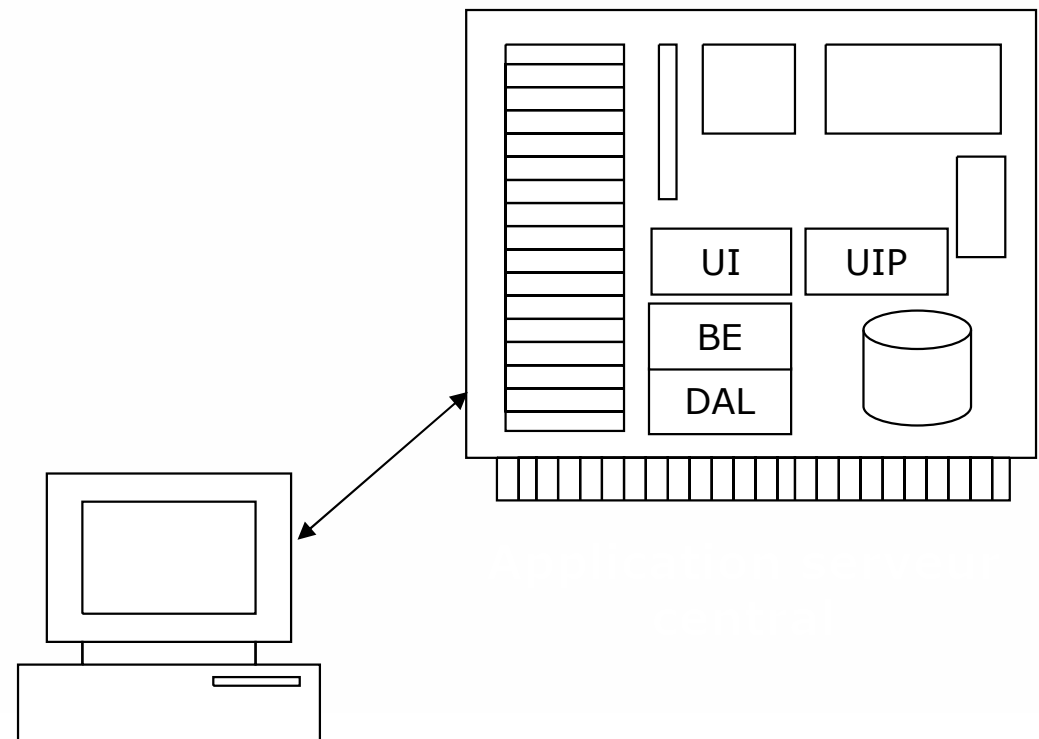
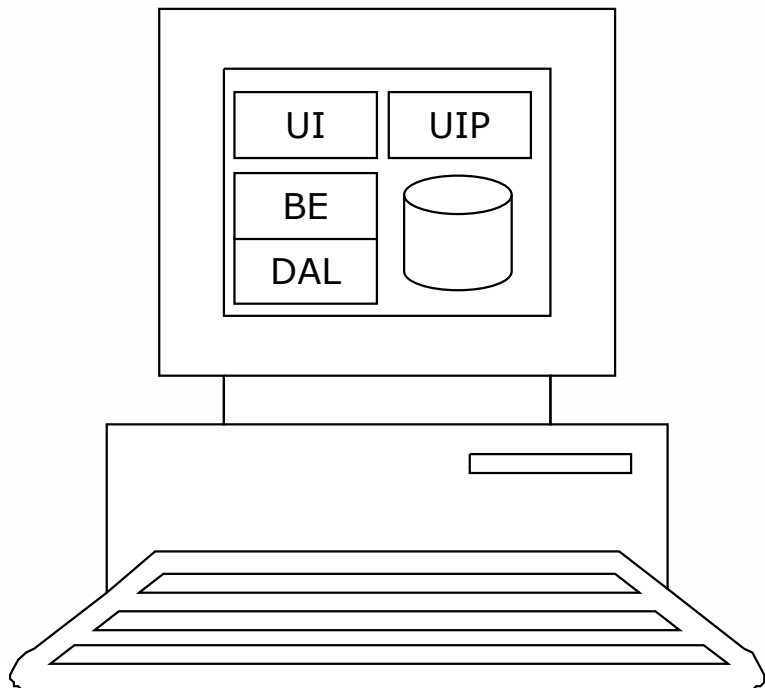
2. Architecture 1-tiers dans le contexte multi utilisateurs

- application sur site central (Mainframe)
- application répartie sur des machines indépendantes communiquant par partage de fichiers.

5.1 Architecture 1-tiers

Il correspond à un seul niveau physique où sont hébergées toutes les couches du système.

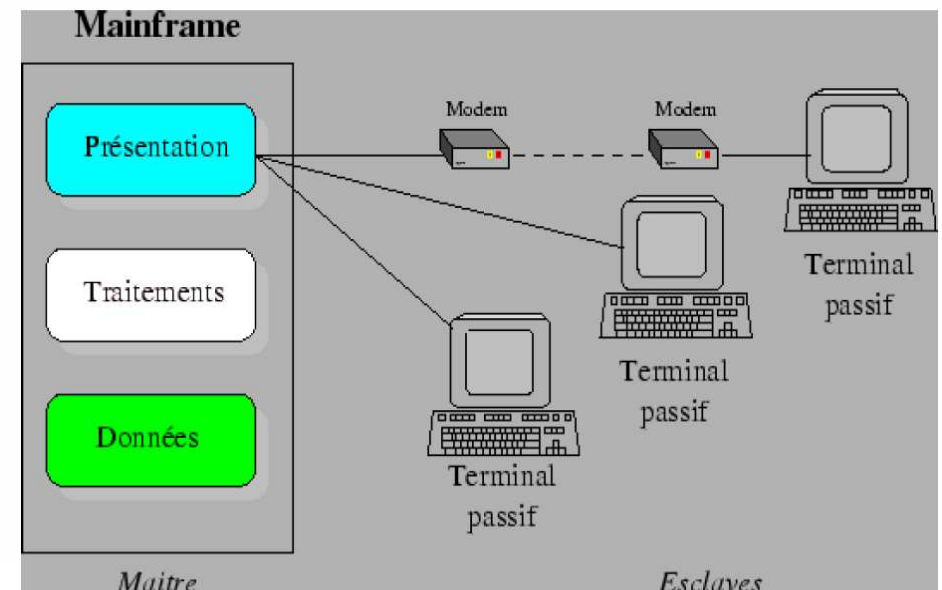
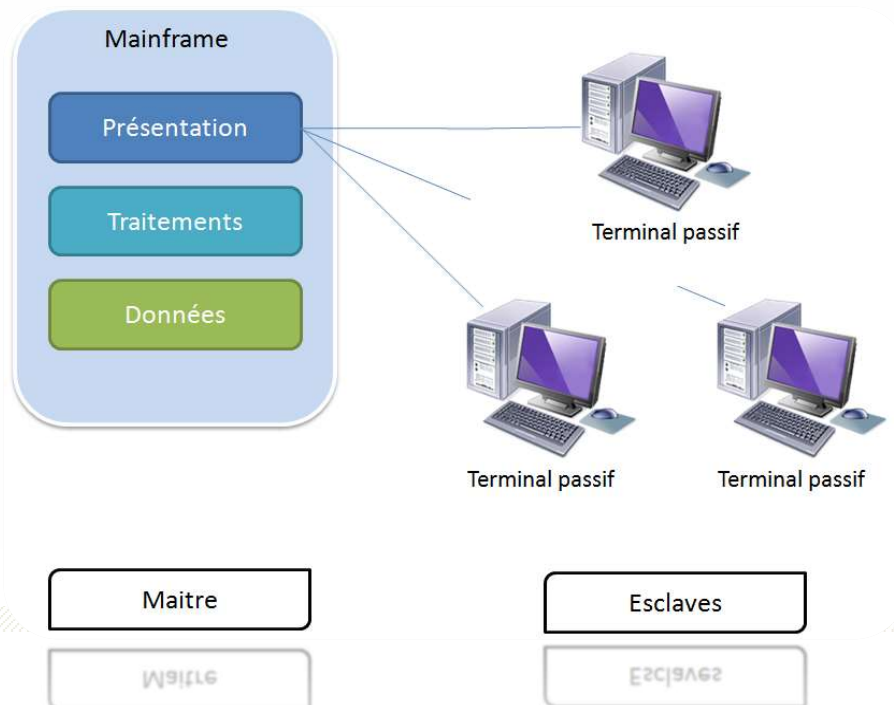
Les applications monopostes ou sur site central sont de ce type.



5.1 Architecture 1-tiers

Application sur Mainframe

- Les utilisateurs se connectent aux applications exécutées par le serveur central (le mainframe) à l'aide de terminaux passifs
- C'est le serveur central qui prend en charge l'intégralité des traitements, y compris l'affichage qui est simplement déporté sur des terminaux passifs.



5.1 Architecture 1-tiers

Les applications un tiers déployées

- Application un tiers sur plusieurs ordinateurs indépendants. Il existe pour cela de nombreux environnements de développement (*dBase, Ms Access, Lotus Approach, Paradox...*).
- Dans un contexte multi-utilisateurs, plusieurs utilisateurs se partagent des fichiers de données stockés sur un serveur commun.
- Le moteur de base de données est exécuté indépendamment sur chaque poste client
- Ce type de solution est donc à réserver à des applications non critiques exploitées par de petits groupes de travail
- La gestion des conflits d'accès aux données doit être prise en charge par chaque programme de façon indépendante, ce qui n'est pas toujours évident.
- La cohabitation de plusieurs moteurs de base de données indépendants manipulant les mêmes données peut devenir assez instable.
 - des conflits lors de la consultation ou de la modification simultanée d'un même enregistrement par plusieurs utilisateurs.
 - Altération de l'intégrité des données.
 - Difficulté d'assurer la confidentialité des données.



5.1 Architecture 1-tiers

■ Avantages

- Mainframe : la fiabilité des solutions sur site central qui gèrent les données de façon centralisée
- Un tiers déployé : l'interface utilisateur moderne des applications.

■ Limites

- Mainframe : interface utilisateur en mode caractères
 - Un tiers déployé : cohabitation d'applications exploitant des données communes peu fiable au-delà d'un certain nombre d'utilisateurs.
-
- Il faut donc trouver une solution conciliant les avantages de cette architecture . Pour se faire, il a fallu scinder les applications en plusieurs parties distinctes et coopérantes :
 - gestion centralisée des données,
 - gestion locale de l'interface utilisateur.
-
- Ainsi est né le concept du **client-serveur**.

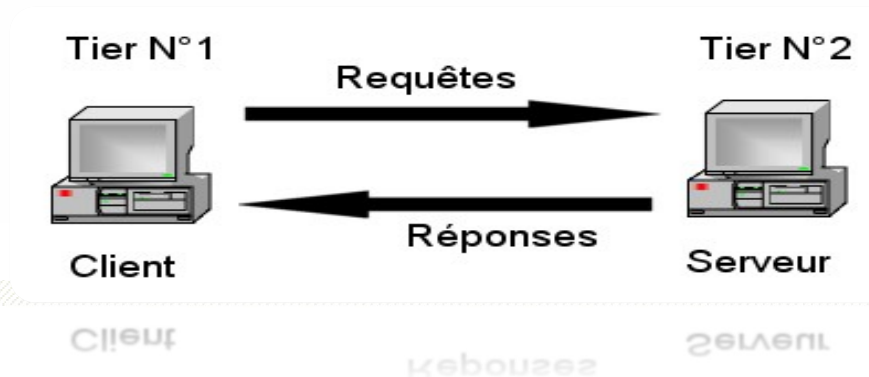
5.2 Architecture 2-tiers

- L'architecture 2-tiers vise à répartir un système informatisé sur des machines distantes, grâce à des matériels et des protocoles standards.
- Il existe deux variantes de l'architecture 2-tiers :
 - Architecture Client/Serveur
 - Architecture Pair-à-Pair

5.2 Architecture 2-tiers

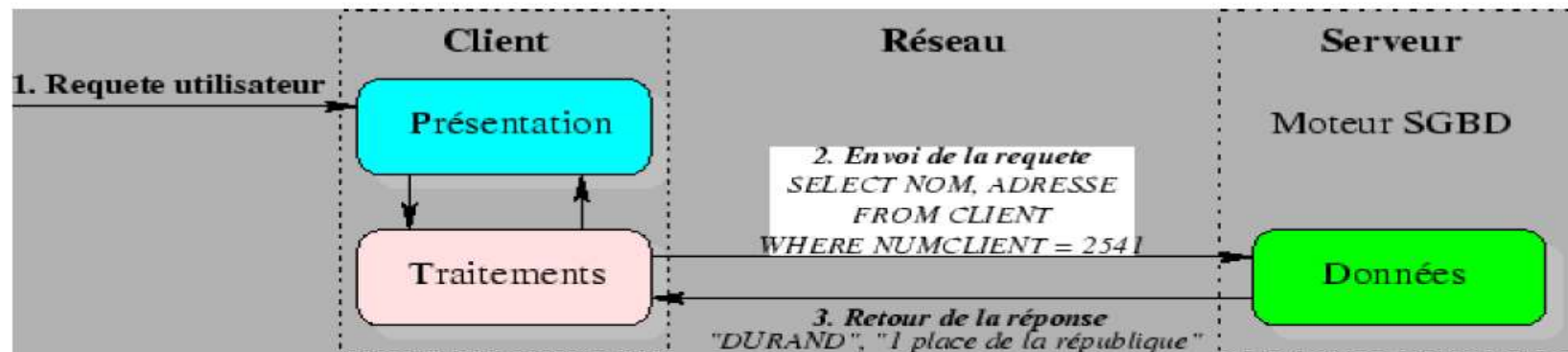
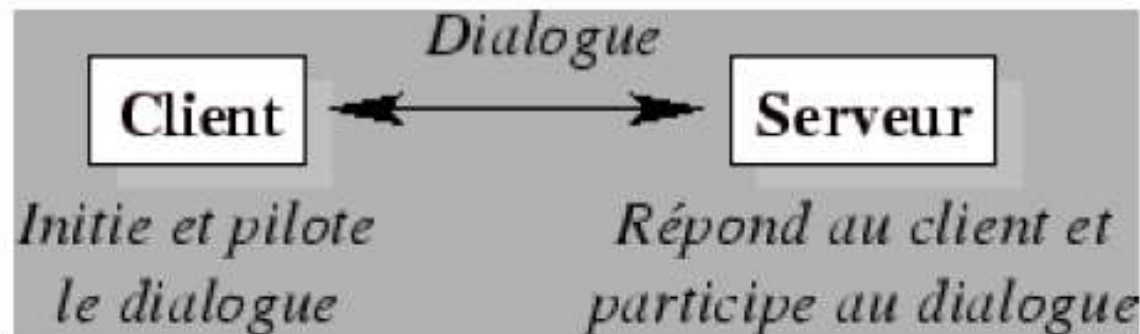
L'architecture **client-serveur** désigne un mode de communication à travers un réseau entre plusieurs logiciels de deux types : l'un, qualifié de **client**, envoie des requêtes ; l'autre, qualifié de **serveur**, attendent les requêtes des clients et y répondent.

Par extension, le client désigne également l'ordinateur sur lequel est exécuté le logiciel client, et le serveur, l'ordinateur sur lequel est exécuté le logiciel serveur.



5.2 Architecture 2-tiers

- Dans une conversation client-serveur, on distingue donc les deux parties suivantes :
 - Le client, c'est le programme qui provoque le dialogue,
 - Le serveur, c'est le programme qui se contente de répondre au client.



5.2 Architecture 2-tiers

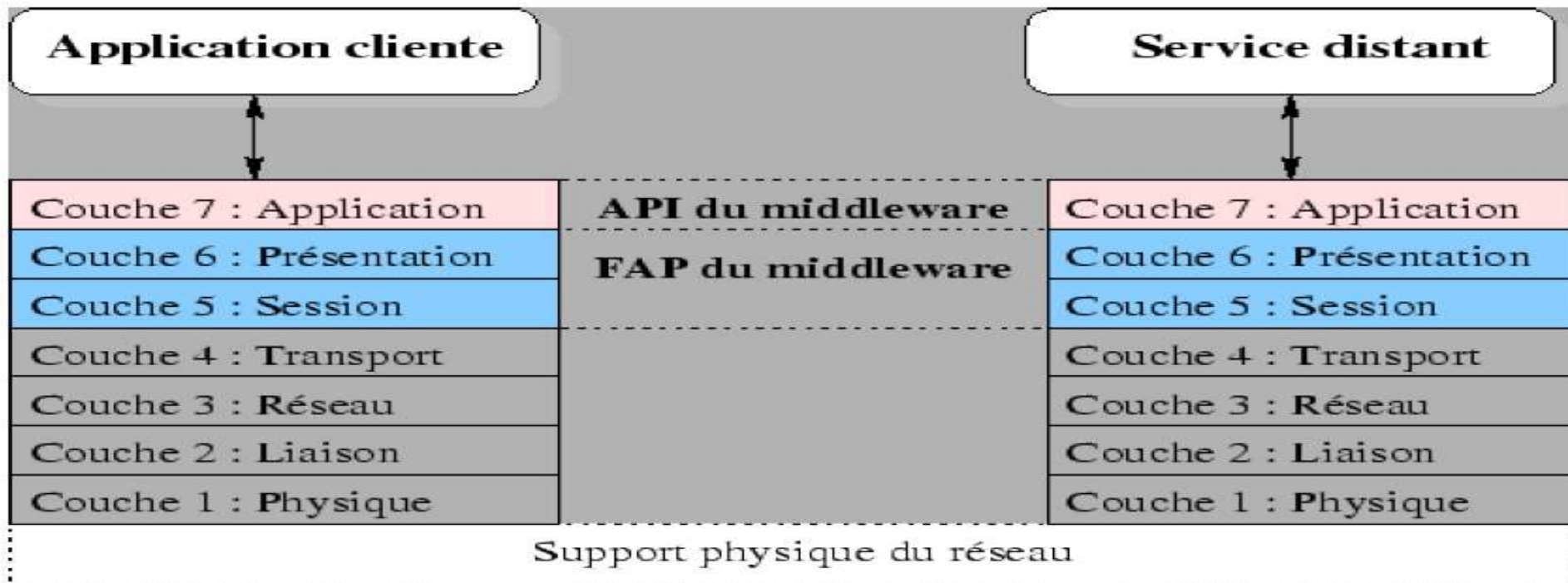
En général, **les serveurs** sont des ordinateurs dotés de capacités supérieures à celles des ordinateurs personnels en termes de puissance de calcul, d'entrées-sorties et de connexions réseau.

Les clients sont souvent des ordinateurs personnels ou des appareils individuels (téléphone, tablette). Un serveur peut répondre aux requêtes d'un grand nombre de clients.

On appelle **middleware**, littéralement ``élément du milieu'', l'ensemble des couches réseau et services logiciel qui permettent le dialogue entre les différents composants d'une application répartie.

- Ce dialogue se base sur un protocole applicatif commun, défini par l'API du *middleware*.
- L'objectif principal du middleware est d'unifier, pour les applications, l'accès et la manipulation de l'ensemble des services disponibles sur le réseau, afin de rendre l'utilisation de ces derniers presque transparente.

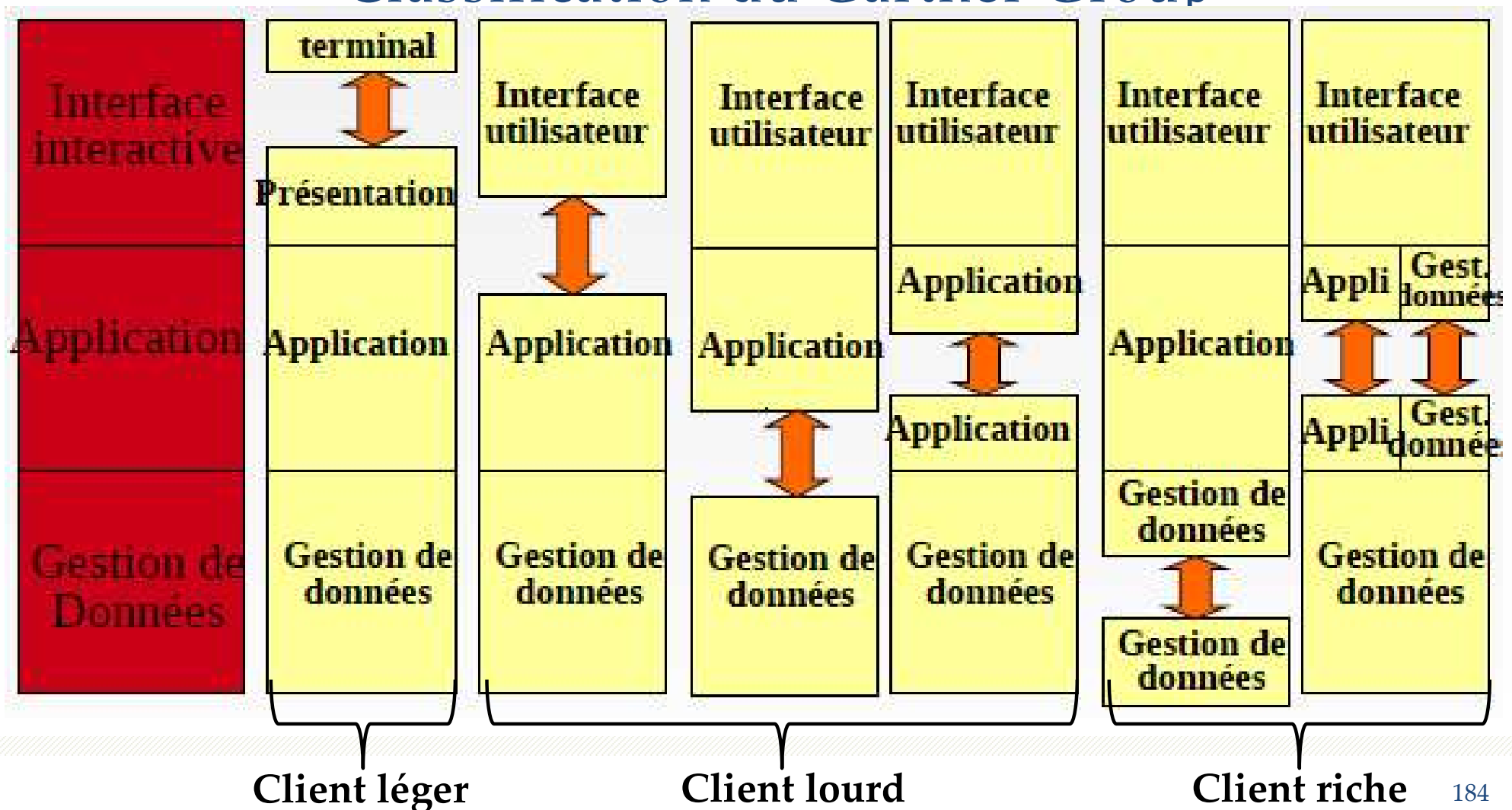
5.2 Architecture 2-tiers



- Middleware = API + FAP
- API : Application Programming Interface (si standard permet la portabilité)
- FAP : Format And Protocols (permet de passer d'un espace d'adressage à un autre et d'une machine à une autre)
- Exemples de middleware:
 - **SQL*Net** : Interface propriétaire permettant de faire dialoguer une application cliente avec une base de données Oracle.
 - **ODBC** : Interface standardisée isolant le client du serveur de données. Elle se compose d'un gestionnaire de driver standardisé, d'une API s'interfaçant avec l'application cliente (sous Ms Windows) et d'un driver correspondant au SGBD utilisé.

5.2 Architecture 2-tiers

Classification du Gartner Group



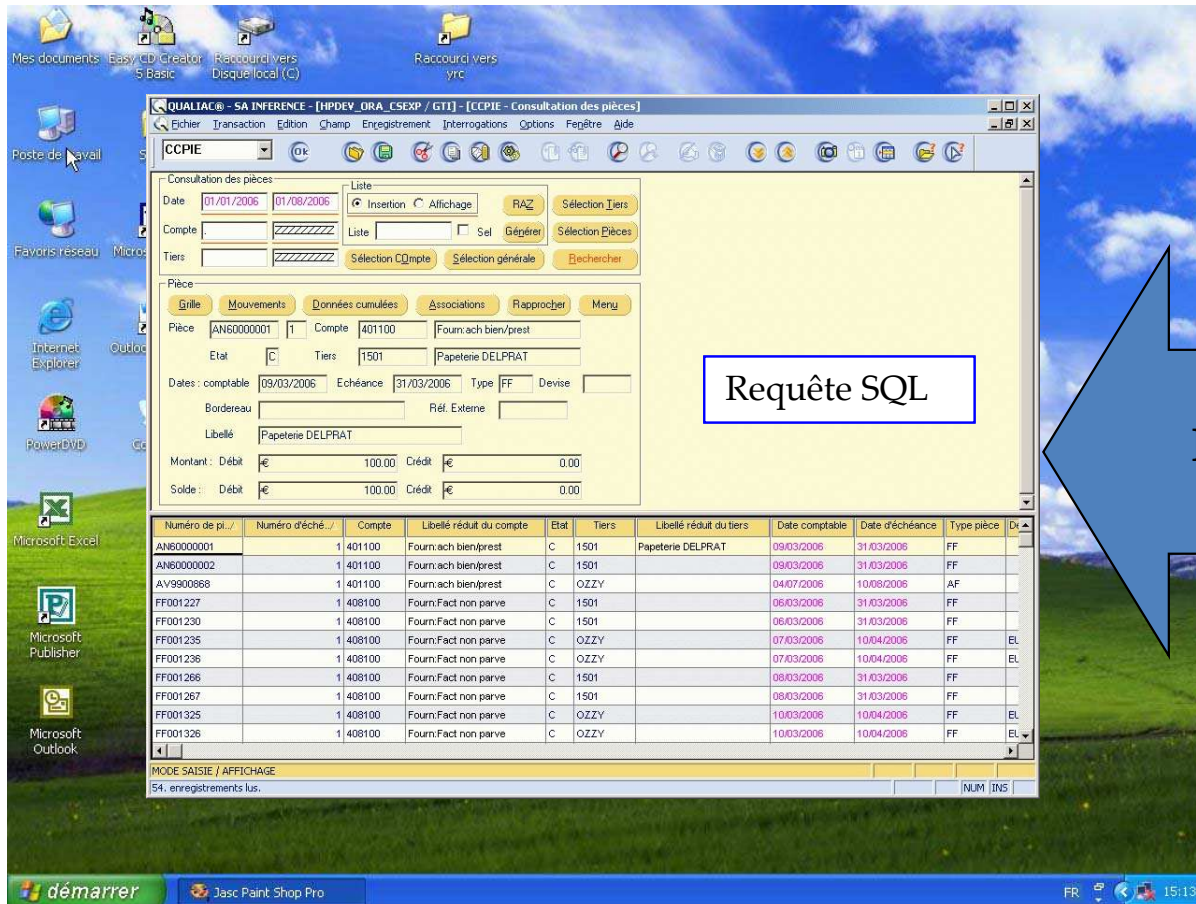
5.2 Architecture 2-tiers

Il existe trois types de clients :

- **Client léger** : L'application fonctionne entièrement sur le serveur, le poste client y accède via un navigateur Web.
- **Client lourd** : Le poste client est capable d'afficher en local des IHM et d'exécuter une partie des traitements.
- **Client riche** : Le poste client est capable d'afficher en local des IHM et d'exécuter une partie des traitements. En plus, il est capable de gérer une partie des données en local.

- Le terme « **client lourd** », en anglais « *fat client* » ou « *heavy client* » par opposition au client léger, désigne une application cliente graphique exécutée sur le système d'exploitation de l'utilisateur.
- Un client lourd possède généralement des capacités de traitement évoluées et peut posséder une interface graphique sophistiquée.
- Néanmoins, ceci demande un effort de développement et tend à mêler la **logique de présentation** (l'interface graphique) avec la **logique applicative** (les traitements)
- On ne peut pas soulager la charge du poste client, qui supporte la grande majorité des traitements applicatifs,
- Le poste client doit être mis à jour régulièrement pour répondre aux besoins des utilisateurs,
- La conversation entre client et serveur est assez bruyante et s'adapte mal à des bandes passantes étroites.
- La relation étroite qui existe entre le programme client et l'organisation de la partie serveur complique les évolutions de cette dernière,
- Ce type d'architecture est grandement rigidifié par les coûts et la complexité de sa maintenance.

Client Lourd



The screenshot shows a Windows XP desktop with a blue sky background. The taskbar includes icons for 'Mes documents', 'Easy CD Creator', 'Raccourci vers Disque local (C:)', 'Raccourci vers lyc', 'Poste de travail', 'Favoris réseau', 'Internet Explorer', 'Outlook', 'PowerDVD', 'Microsoft Excel', 'Microsoft Publisher', and 'Microsoft Outlook'. The application window is titled 'QUALIAC@ - SA INFERENCE - [HPDEV_DRA_CSEXP / GTI] - [CCPIE - Consultation des pièces]'. It features a menu bar (Fichier, Transaction, Edition, Champ, Enregistrement, Interrogations, Options, Fenêtre, Aide) and a toolbar. The main window is divided into several sections: 'Consultation des pièces' with filters for Date (01/01/2006 to 01/08/2006), Compte, and Tiers; a 'Pièce' section with tabs for Grille, Mouvements, Données cumulées, Associations, Rapprocher, and Menu; and a table of data. The table has columns for Numéro de pi., Numéro d'éché., Compte, Libellé réduit du compte, Etat, Tiers, Libellé réduit du tiers, Date comptable, Date d'échéance, Type pièce, and Dr. The status bar at the bottom indicates 'MODE SAISIE / AFFICHAGE' and '154. enregistrements lus.'.

Requête SQL

IPC/SGBD

Data

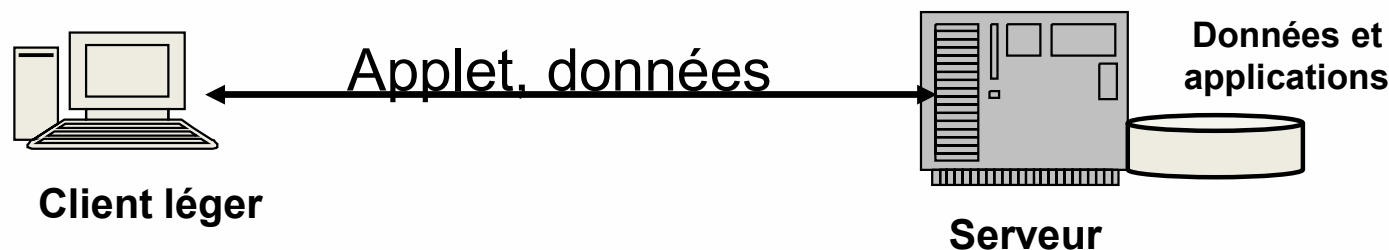
Data

CLIENT LÉGER

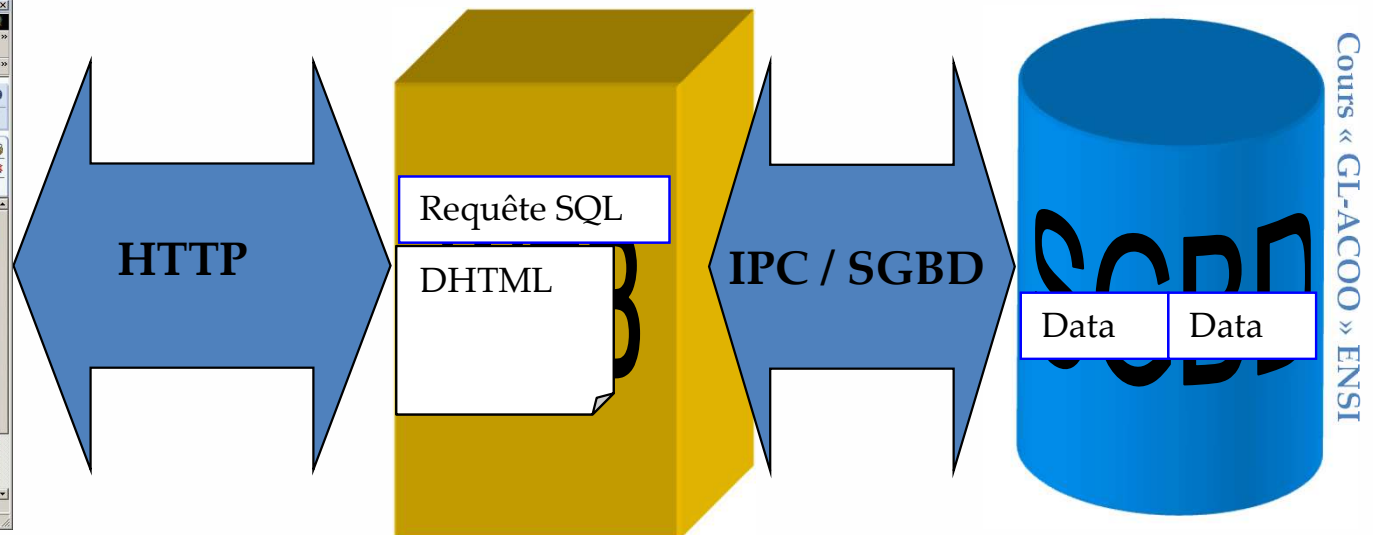
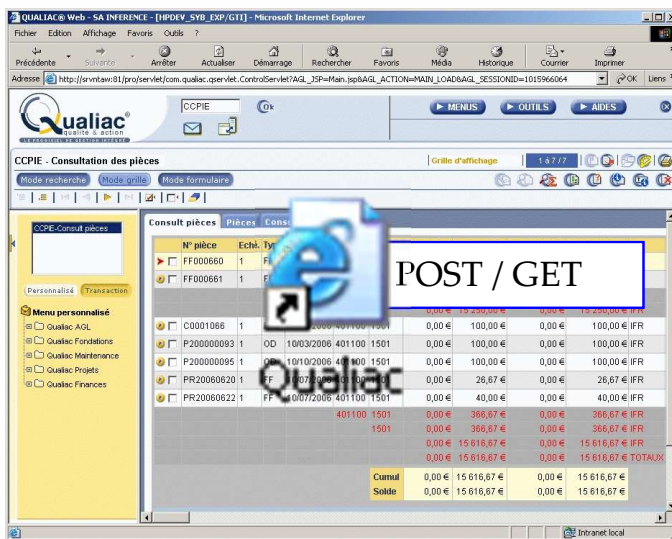


Section 5 Les styles physiques

- Le terme « **client léger** », parfois « *client pauvre* », en anglais « *thin client* » par opposition au client lourd.
- Client à fonctionnalité minimale: Terminaux X, stations de travail sans disque dur, Ordinateur Réseau (Network Computer), Ordinateur en réseau (Networked PC),...
- Beaucoup de charge sur le serveur
- Désigne une application accessible via une interface web (en HTML) consultable à l'aide d'un navigateur web, où la totalité de la logique métier est traitée du côté du serveur. Pour ces raisons, le navigateur est parfois appelé **client universel**.
- L'origine du terme provient de la limitation du langage HTML, qui ne permet de faire des interfaces relativement pauvres en interactivité, si ce n'est pas le biais du langage javascript, DHTML, XHTML, etc..



Client Léger



- Un « **client riche** » est un compromis entre le client léger et le client lourd.
- L'objectif du client riche est donc de :
 - proposer une interface graphique, décrite avec une grammaire de description basée sur la syntaxe XML,
 - obtenir des fonctionnalités similaires à celles d'un client lourd (glisser déposer, onglets, multi fenêtrage, menus déroulants, ...).
- Il existe des standards permettant de définir une application riche :
 - **XAML** (*eXtensible Application Markup Language*), un standard XML proposé par Microsoft ;
 - **XUL**, un standard XML proposé par la fondation Mozilla ;
 - **Flex**, un standard XML proposé par la société Macromedia

Navigateur + Plug-in

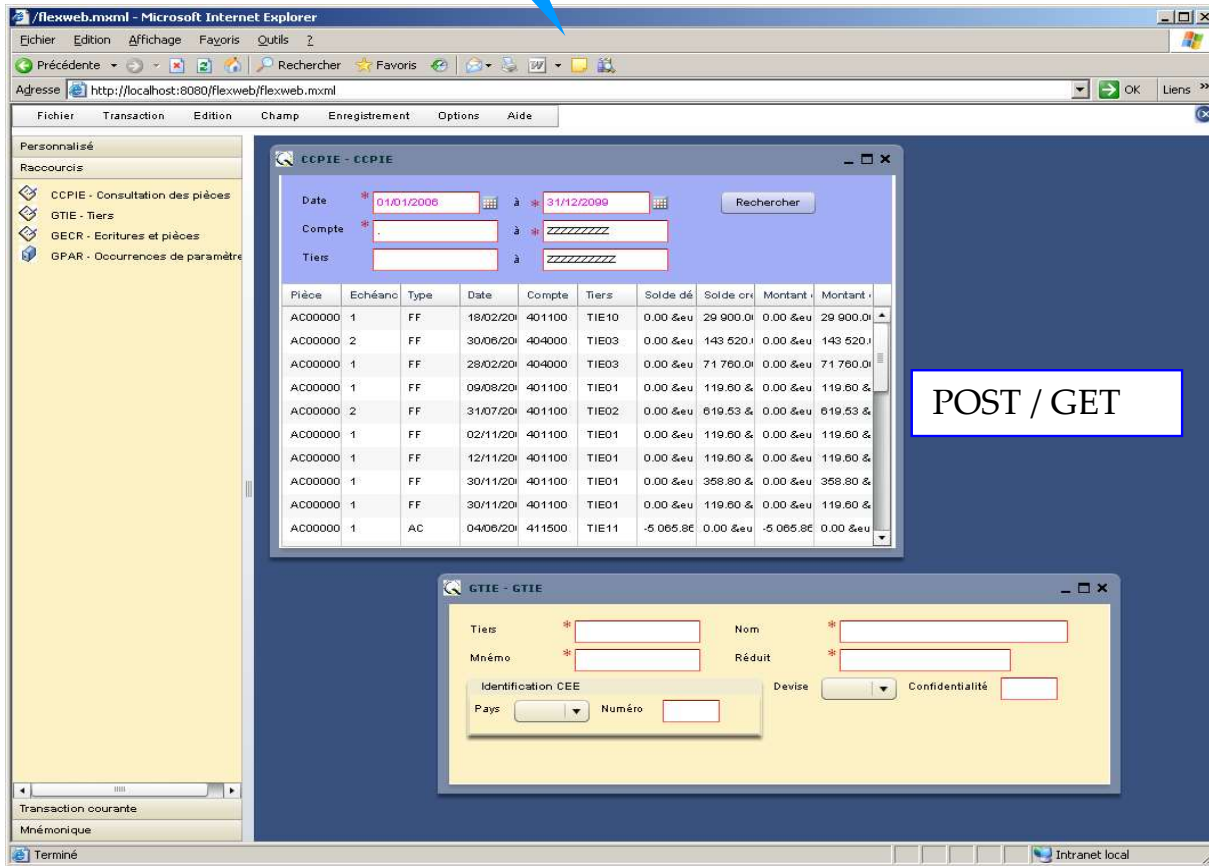


HTTP ou SOAP

CODE
APPLICATION

Client Riche

Navigateur + Plug-in



The screenshot shows a Microsoft Internet Explorer window displaying a web application. The address bar shows 'http://localhost:8080/flexweb/flexweb.xml'. The application has a sidebar with a 'Personnalisé' menu containing 'Raccourcis' and 'CCPIE - Consultation des pièces'. The main content area displays a table of transactions with columns: Pièce, Echéanc, Type, Date, Compte, Tiers, Solde ds, Solde cr, Montant, and Montant. Below the table is a 'GTIE - GTIE' form with fields for Tiers, Nom, Mném, Réduit, Identification CEE, Pays, Numéro, Devise, and Confidentialité.

Pièce	Echéanc	Type	Date	Compte	Tiers	Solde ds	Solde cr	Montant	Montant
AC00000	1	FF	18/02/20	401100	TIE10	0.00 &eu	29 900.00	0.00 &eu	29 900.00
AC00000	2	FF	30/08/20	404000	TIE03	0.00 &eu	143 520.00	0.00 &eu	143 520.00
AC00000	1	FF	28/02/20	404000	TIE03	0.00 &eu	71 760.00	0.00 &eu	71 760.00
AC00000	1	FF	09/08/20	401100	TIE01	0.00 &eu	119.60 &	0.00 &eu	119.60 &
AC00000	2	FF	31/07/20	401100	TIE02	0.00 &eu	619.53 &	0.00 &eu	619.53 &
AC00000	1	FF	02/11/20	401100	TIE01	0.00 &eu	119.60 &	0.00 &eu	119.60 &
AC00000	1	FF	12/11/20	401100	TIE01	0.00 &eu	119.60 &	0.00 &eu	119.60 &
AC00000	1	FF	30/11/20	401100	TIE01	0.00 &eu	358.80 &	0.00 &eu	358.80 &
AC00000	1	FF	30/11/20	401100	TIE01	0.00 &eu	119.60 &	0.00 &eu	119.60 &
AC00000	1	AC	04/06/20	411500	TIE11	-5 065.86	0.00 &eu	-5 065.86	0.00 &eu

POST / GET

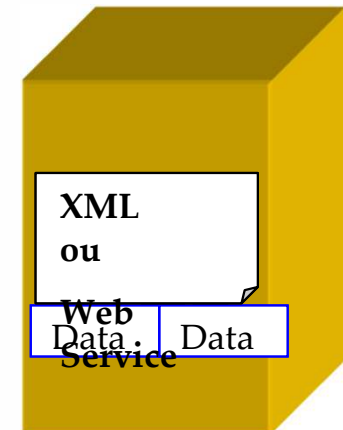
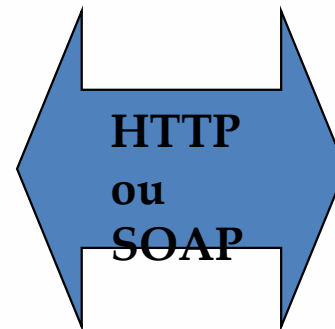
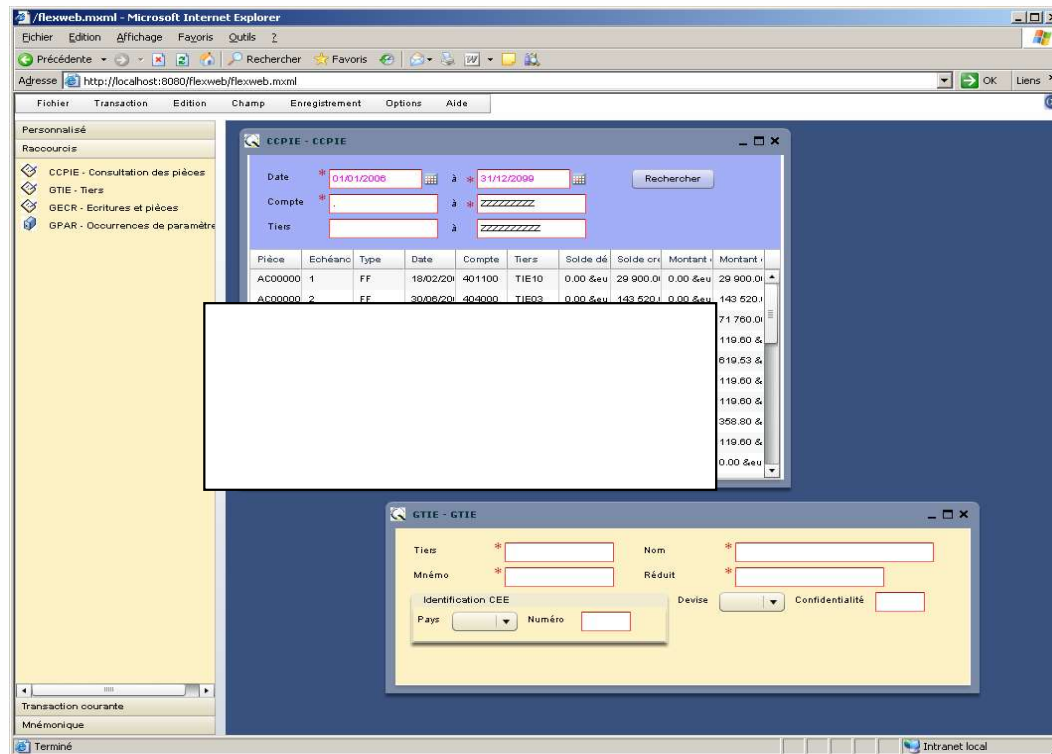
HTTP
ou
SOAP

Requête SQL

WEB

Client Riche

Navigateur + Plug-in



5.2 Architecture 2-tiers

- Clients lourds
 - Traitement évolué
 - Grande ergonomie
 - Problème de mises à jour
- Clients légers
 - Traitement coté serveur
 - Mises à jour aisées
 - Interface graphique pauvre
- Clients riches
 - Compromis lourd - léger
- **Points forts**
 - Couplage faible (le serveur n'a pas besoin des clients)
 - Maintenance et administration du serveur simplifiées
 - Souplesse : possibilité d'ajouter/supprimer dynamiquement des clients sans perturber le fonctionnement de l'application
 - Les ressources sont centralisées sur le serveur (pas de problème d'intégrité/cohérence des données)
 - Sécurisation simple des données
- **Points faibles**
 - Un maillon faible : problème de performance/disponibilité pour le serveur
 - Coût élevé : le serveur doit être très performant pour honorer tous les clients

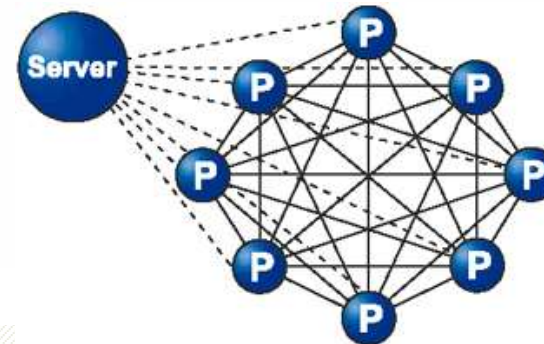
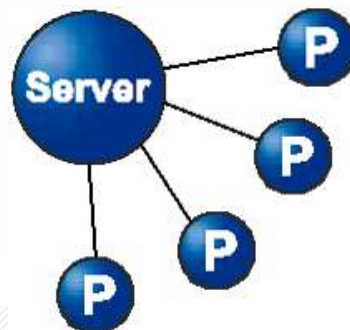
5.2 Architecture 2-tiers

- L'architecture **pair à pair, égal à égal ou point à point** (*peer-to-peer*, souvent abrégé « P2P ») est une variante d'architecture client-serveur où chaque client est aussi un serveur.
- On appelle souvent **servent** (**serveur** + **client**) un nœud (ordinateur) dans une architecture pair-à-pair qui se base sur la fusion des fonctions de client et de serveur .

5.2 Architecture 2-tiers

Des propositions utilisant le modèle pair-à-pair ont été faites pour ne plus utiliser de serveurs (qui font tout à la fois), entre autres :

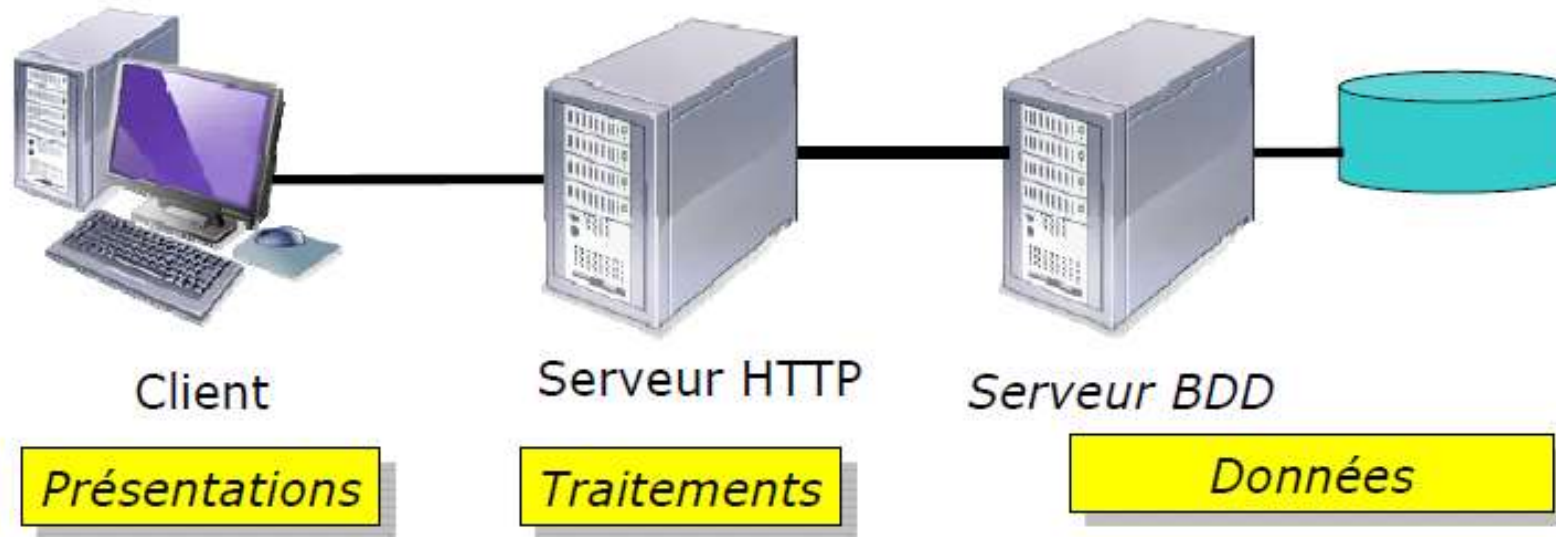
- Les systèmes de streaming (diffuser des contenus multimédias en « direct ») ;
- les logiciels de messagerie en ligne ;
- Service collaboratif d'édition de documents
- Les systèmes de partage de fichiers (le plus connu Bittorrent)



5.3 Architecture 3-tiers

Architecture 3 tiers

- La présentation et les traitements sont pris en charge par le client et/ou le serveur intermédiaire (selon le type de client)
- Les données sont gérées par un **serveur de données**

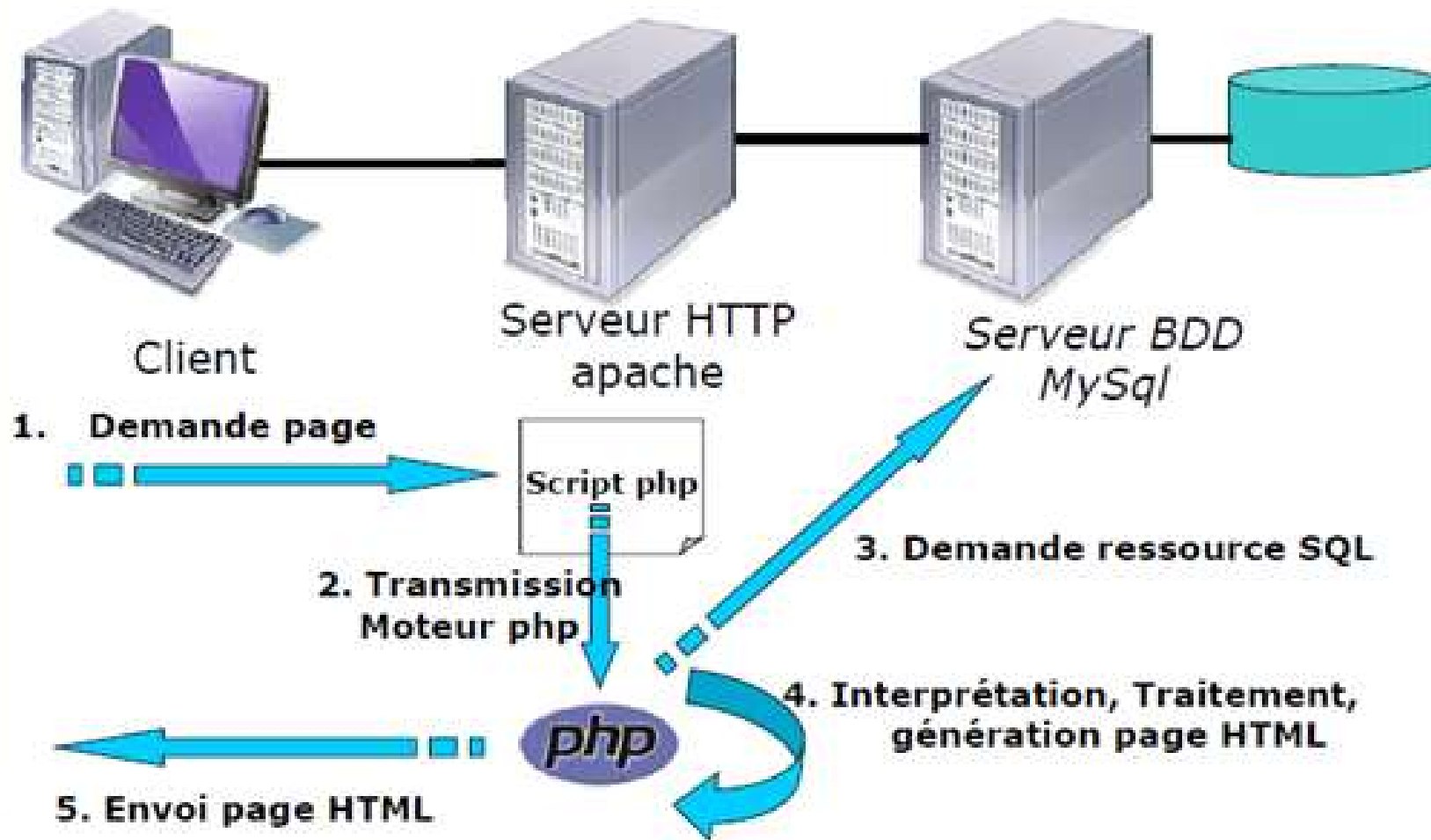


5.3 Architecture 3-tiers

- Les données sont toujours gérées de façon centralisée
- La présentation est toujours prise en charge par le poste client
- La logique applicative est prise en charge par un serveur intermédiaire
- Tous ces niveaux étant indépendants, ils peuvent être implantés sur des machines différentes, de ce fait :
 - Le poste client ne supporte plus l'ensemble des traitements (client léger)
 - Facilité de déploiement
 - Sécurité : pas d'exposition du schéma de la base de données
 - La manipulation des données est indépendante du support physique de stockage
- Il est relativement simple de faire face à une forte montée en charge, en renforçant le service applicatif.

5.3 Architecture 3-tiers

Architecture 3 tiers – Exemple PHP/MySql



5.3 Architecture 3-tiers

Exemples des tiers :

- **Le tiers client :**

- Un navigateur Web (google chrome, Internet explorer, fireFox...)
- Un PDA ou SmartPhone

- **Le tiers Applicatif :**

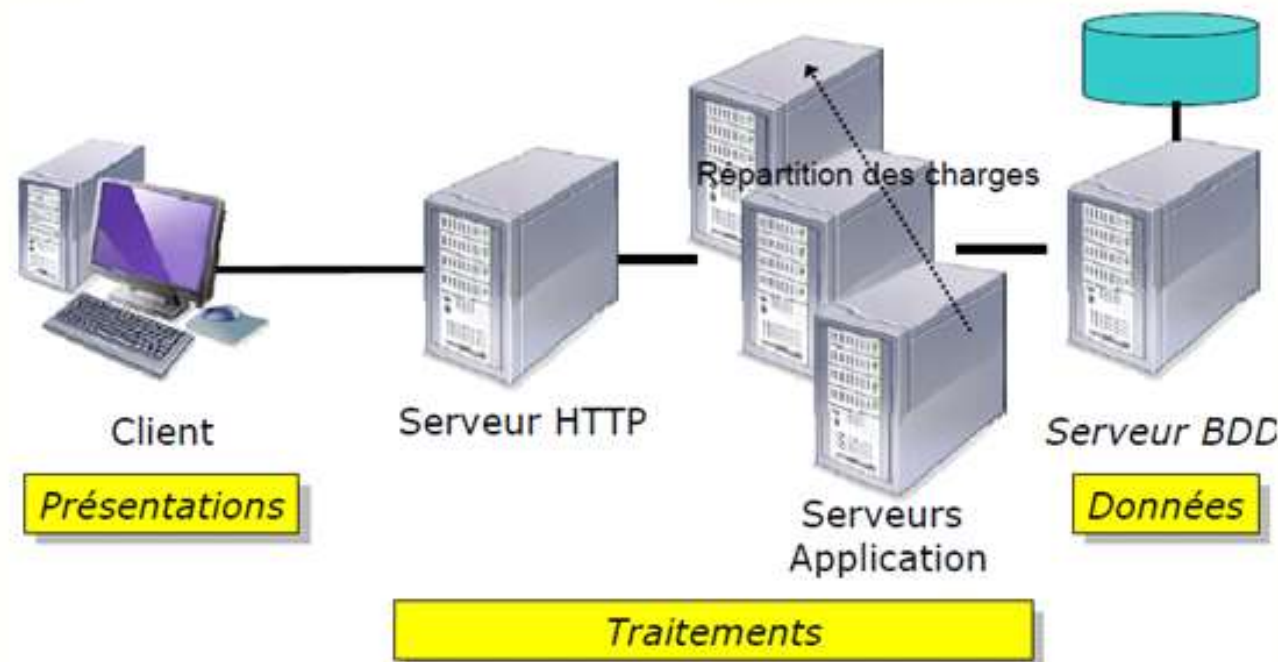
- Serveur web (tomcat, WampServer...)

- **Le tiers données :**

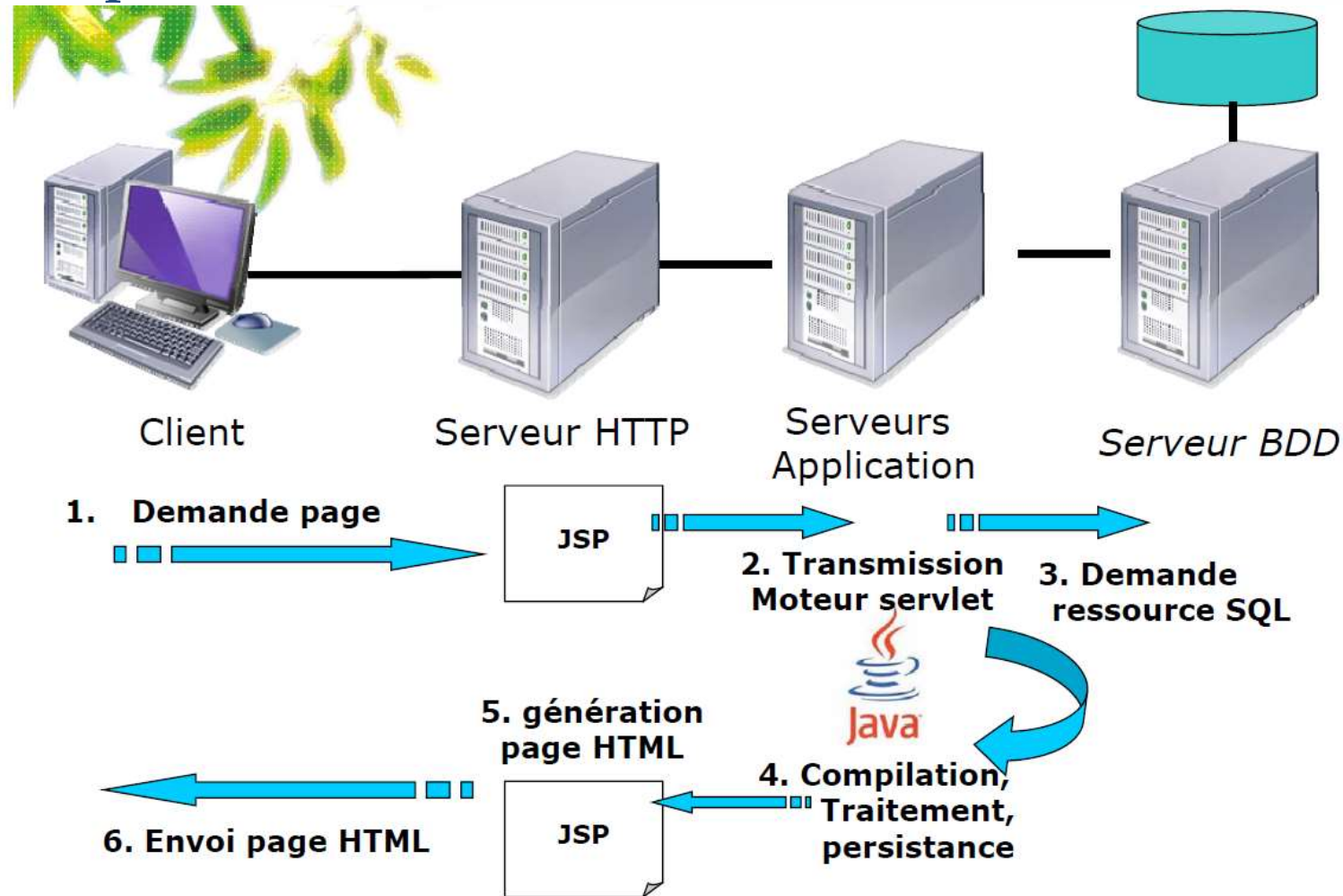
- SGBD (MySQL, WampServer, MS Access...)

5.4 Architecture N-tiers

- **Architecture N tiers** ou aussi architecture multi-tiers : elle qualifie la **distribution de traitements** entre de multiples serveurs d'application (qui peuvent être hétérogènes) ou la **distribution de données** entre plusieurs serveurs de données.



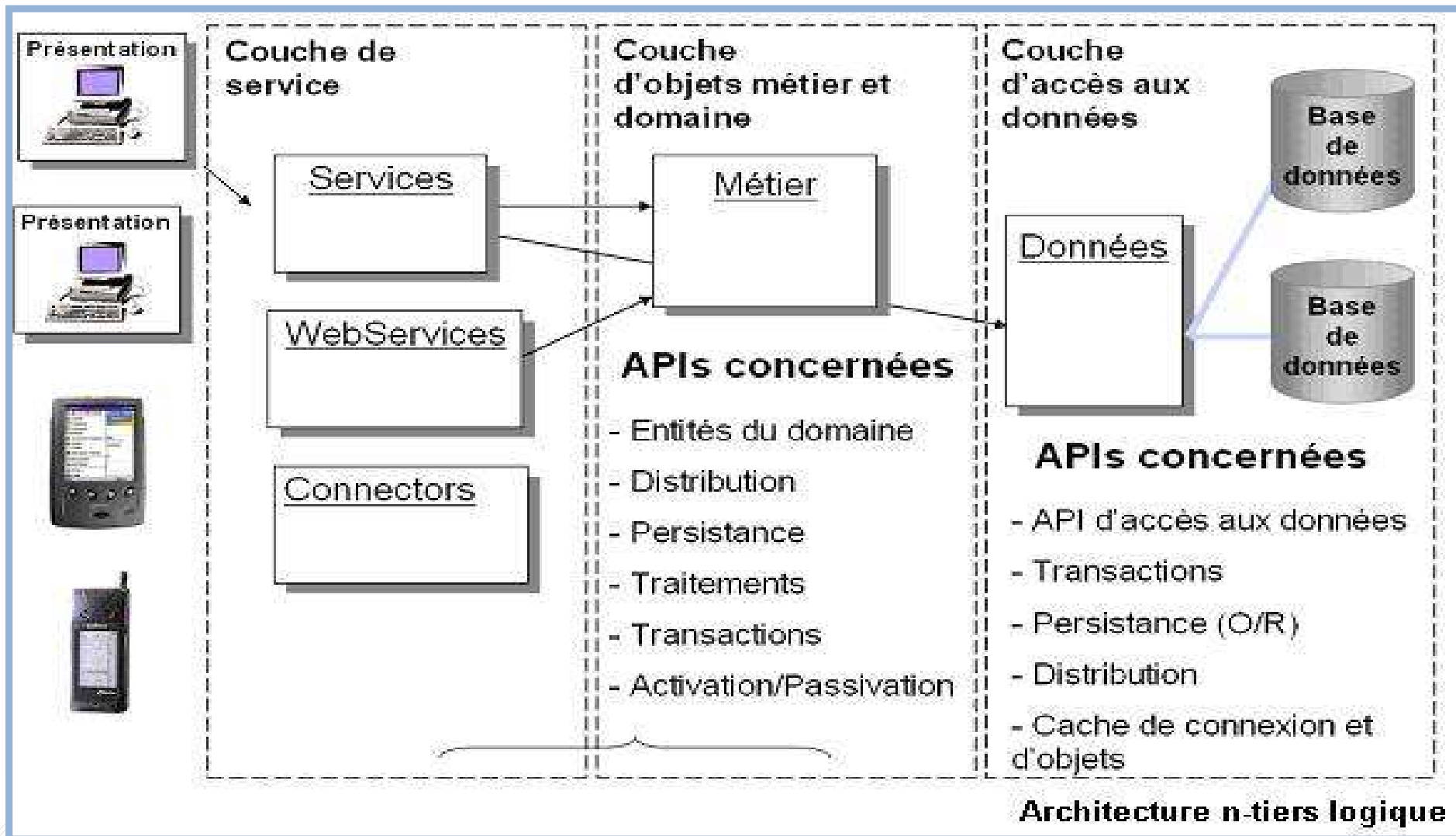
■ Exemple: 4-tiers:



5.4 Architecture N-tiers

- L'appellation ``N-tiers'' pourrait faire penser que cette architecture met en œuvre un nombre indéterminé de niveaux de service, alors que ces derniers sont au maximum trois.
- En fait, l'architecture n-tiers qualifie la distribution d'application entre de multiples services et non la multiplication des niveaux de service.
- Cette distribution est facilitée par l'utilisation de composants ``métier'', spécialisés et indépendants, introduits par les concepts orientés objets
- Elle permet de tirer pleinement partie de la notion de composants métiers réutilisables.
- Ces composants rendent un service si possible générique et clairement identifié. Ils sont capables de communiquer entre eux et peuvent donc coopérer en étant implantés sur des machines distinctes.
- La distribution des services applicatifs facilite aussi l'intégration de traitements existants dans les nouvelles applications.

5.4 Architecture N-tiers



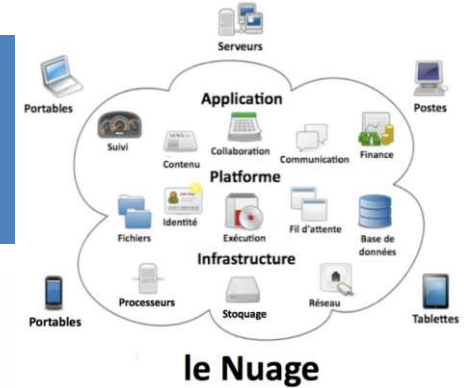
Avantages

- Séparation poussée des tâches
- Permet de supporter un grand nombre de formats de présentation différents (propres à chaque client), tout en réutilisant certains des objets de présentation entre les clients.

Inconvénients

- Nécessite de ressources matérielles importantes

CLOUD COMPUTING



- Le Cloud Computing est une technologie basée sur *internet* qui permet de fournir des services (ressources matérielles et/ou logicielles) d'une manière *évolutive* par des fournisseurs.
- Le Cloud Computing décharge le client de l'infrastructure IT puisqu'il fournit le matériel et l'infrastructure
- Les utilisateurs ne se soucient plus de l'évolution et de la maintenance des logiciels
- Le cloud permet de personnaliser les architectures des systèmes à déployer en s'appuyant sur *plusieurs styles*: SOA, multi-couches, N-tiers
- La plupart des systèmes de grande échelle sont hétérogènes et ne correspondent pas à un style architectural unique



Les trois couches

Logiciel sous la forme d'un Service (Software as a Service)

- logiciels finaux
- avantages :
 - financiers
 - maintenance/sécurité
 - collaboration



Plateforme sous la forme d'un Service (Platform as a Service)

- outils de développement
- outils de test et de maintenance
- outils de déploiement



Infrastructure sous la forme d'un Service (Infrastructure as a Service)

- ressources physiques à la demande
- duplication des données
- montée en charge



- Le Cloud Computing permet aux entreprises une *réduction des coûts* car le client ne paye que le stockage et l'utilisation des processeurs
- Une structure basée sur le cloud est *théoriquement infaillible* car lorsque les serveurs actuels ne peuvent plus répondre à la demande, un nouveau serveur virtuel est automatiquement créé
- Exemples de fournisseurs cloud:
 - Google AppEngine
 - Amazon
 - Windows Azure

CONCLUSION

- Comme la modélisation, le design est une habilité qui s'acquière avec l'expérience
- Un logiciel mal conçu mène à une maintenance coûteuse
 - Toujours s'assurer de suivre les principes de conception
- La qualité d'un design exige une attention de tous les instants
 - Concevoir le design de façon flexible afin de s'adapter harmonieusement aux changements.
 - S'assurer que la documentation est adéquate
 - S'assurer que les changements sont bien gérés
- Utiliser les styles architecturaux
- Utiliser les patrons de conception